คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

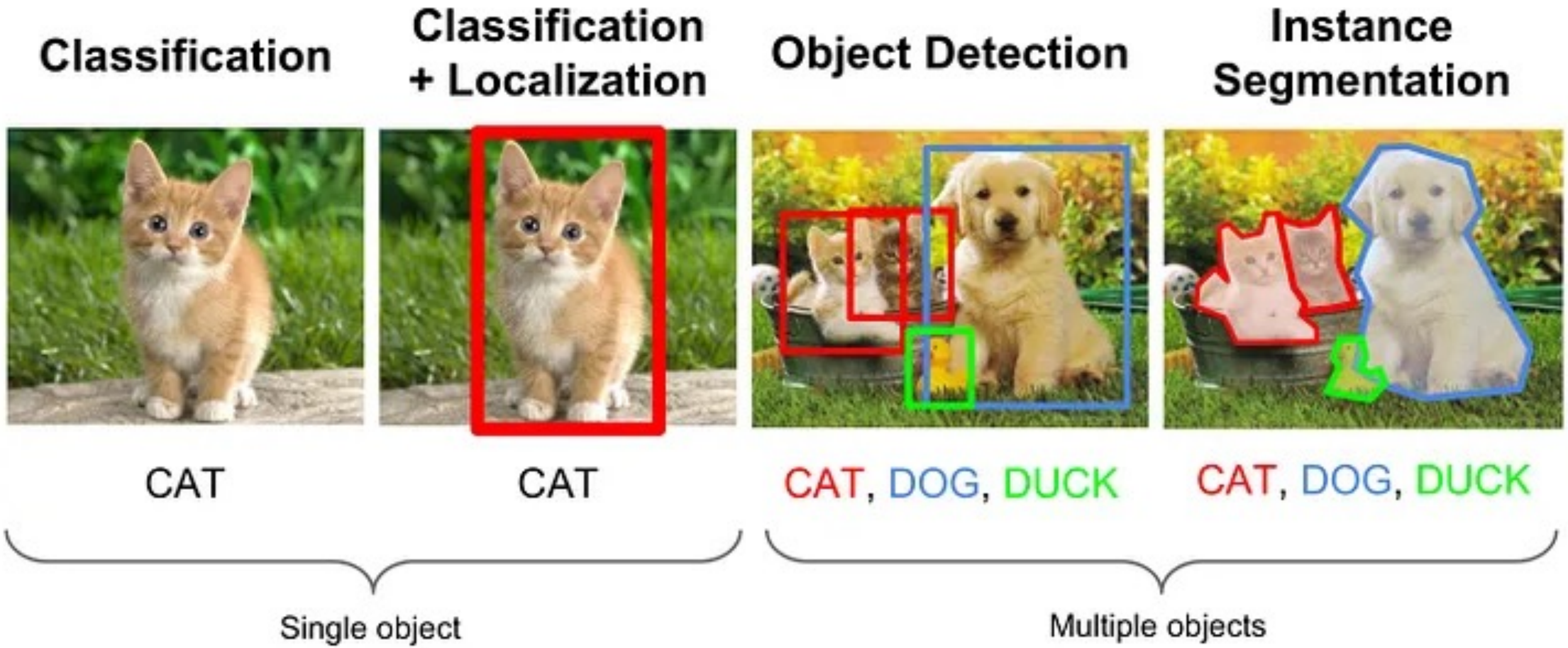
# Chapter 11: Object Detection with Deep Learning Techniques

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

# ความแตกต่างระหว่าง
# Image Classification และ Object Detection





# หัวข้อ

- **Multi-Stage Detectors**
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- **Single-Stage Detectors**
  - You Only Look Once (YOLO)
- **การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics**

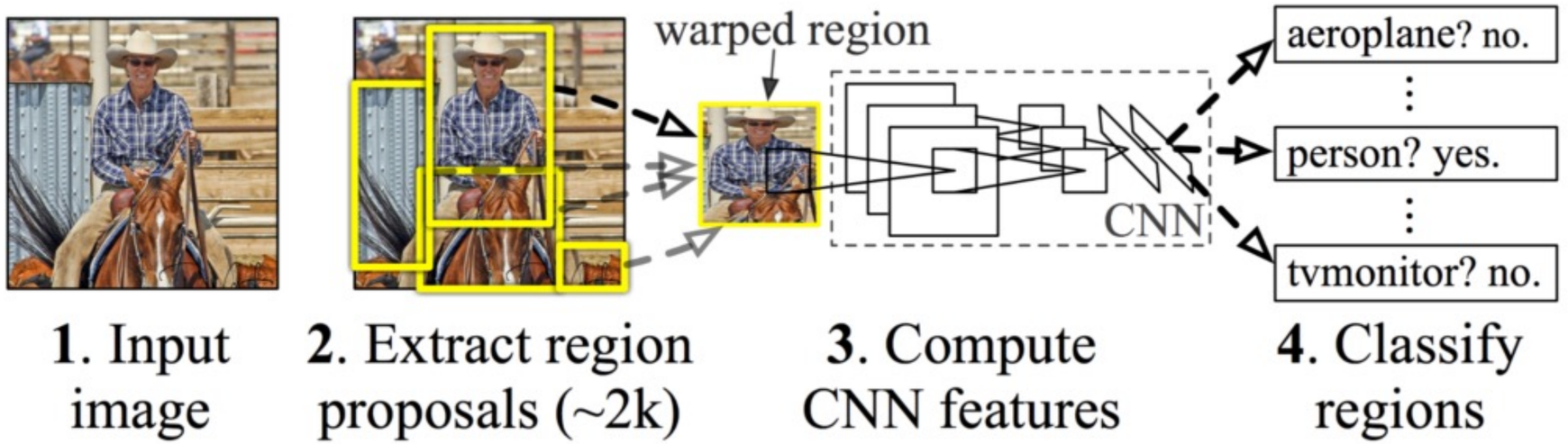


# หัวข้อ

- Multi-Stage Detectors
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- Single-Stage Detectors
  - You Only Look Once (YOLO)
- การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics

# Region-based Convolutional Network (R-CNN)

เป็นเครื่องหนึ่งในกระบวนการเรียนรู้เชิงลึกสำหรับงานทางด้าน Computer Vision และ Image Processing ที่ได้รับการออกแบบมาเพื่อการตรวจจับวัตถุในภาพ ซึ่งได้รับการคิดค้นโดย Ross Girshick และคณะในปี 2014 วิธีการ R-CNN ประกอบไปด้วย 4 ขั้นตอน ได้แก่ (1) การตรวจจับวัตถุในภาพ (2) การปรุงปรับภาพ (3) การสกัดคุณลักษณะ และ (4) การจำแนกภาพ



# Region-based Convolutional Network (R-CNN)

**ขั้นตอนที่ 1** การตรวจจับวัตถุในภาพด้วยหลักการ Selective Search เป็นการพิจารณาพิกเซลที่ติดกันว่ามีความคล้ายคลึงกันหรือไม่ โดยพิจารณาจาก color, texture, size และ shape compatibility



# Region-based Convolutional Network (R-CNN)

**ขั้นตอนที่ 1.1** พิจารณาพิกเซลที่ติดกันตามหลักการ graph-based segmentation ที่ได้คิดค้นโดย Felzenszwalb และคณะ ผลลัพธ์ที่ได้จะเรียกว่า Region Proposal

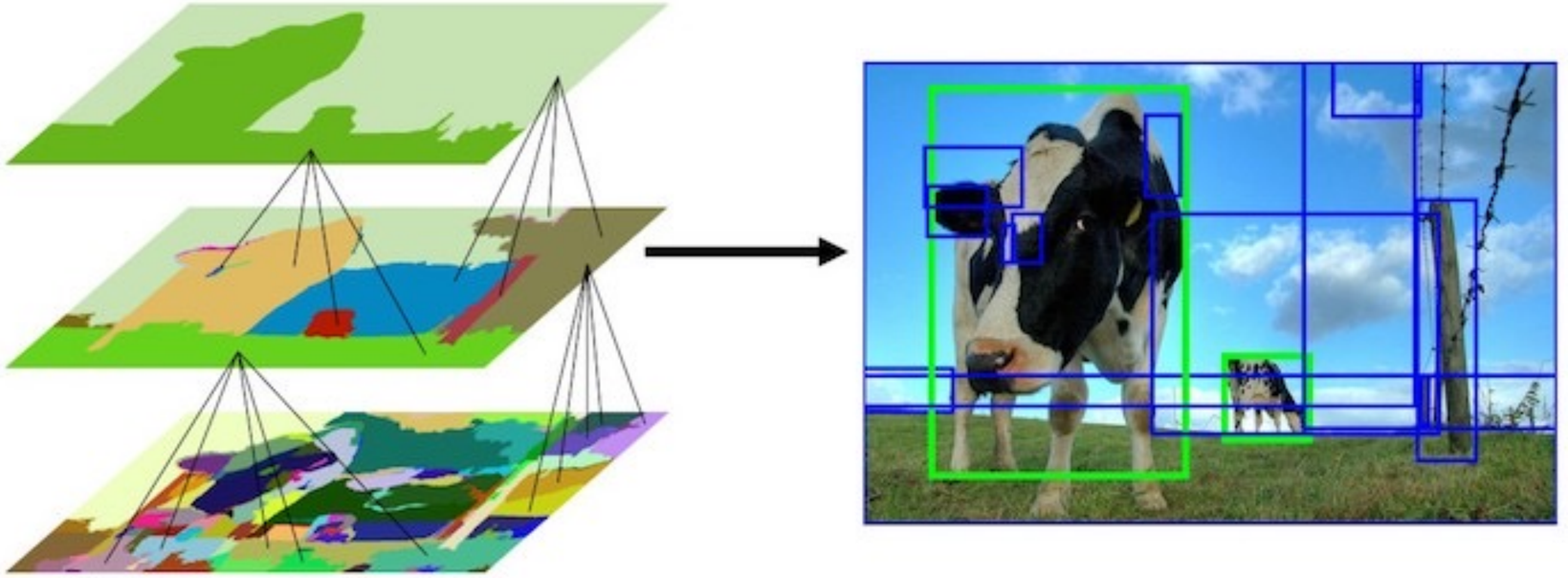






อ้างอิง https://learnopencv.com/selective-search-for-object-detection-cpp-python/

# Region-based Convolutional Network (R-CNN)



**ขั้นตอนที่ 1.2** เก็บทุก Regional Proposal ไว้ใน List จากนั้น ให้จัดกลุ่มหรือรวม Regional Proposal ที่ติดกันเข้าด้วยกันโดยอาศัยการวัดความคล้ายจาก color, texture, size และ shape compatibility ดังสมการต่อไปนี้

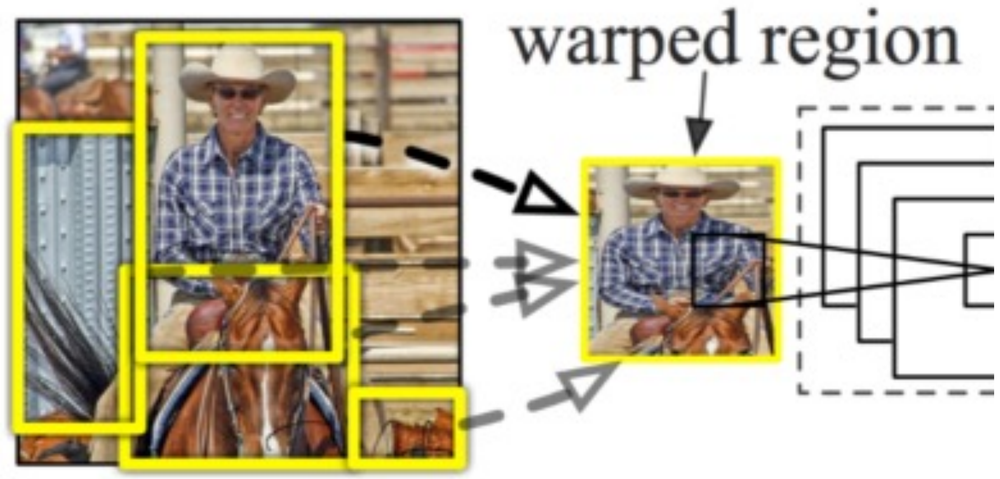
$$s(r_i, r_j) = a_1 s_{color}(r_i, r_j) + a_2 s_{texture}(r_i, r_j) + a_3 s_{size}(r_i, r_j) + a_4 s_{fill}(r_i, r_j)$$

โดยที่ $r_i$ และ $r_j$ คือ สอง Regional Proposal ที่ติดกัน และ $a_i$ ใช้กำหนดน้ำหนักตัวชี้วัดที่ต้องการ (ค่าเป็น 0 หรือ 1) ซึ่งจะได้ผลลัพธ์ประมาณ 1,000 ถึง 2,000 Proposal

อ้างอิง https://learnopencv.com/selective-search-for-object-detection-cpp-python/

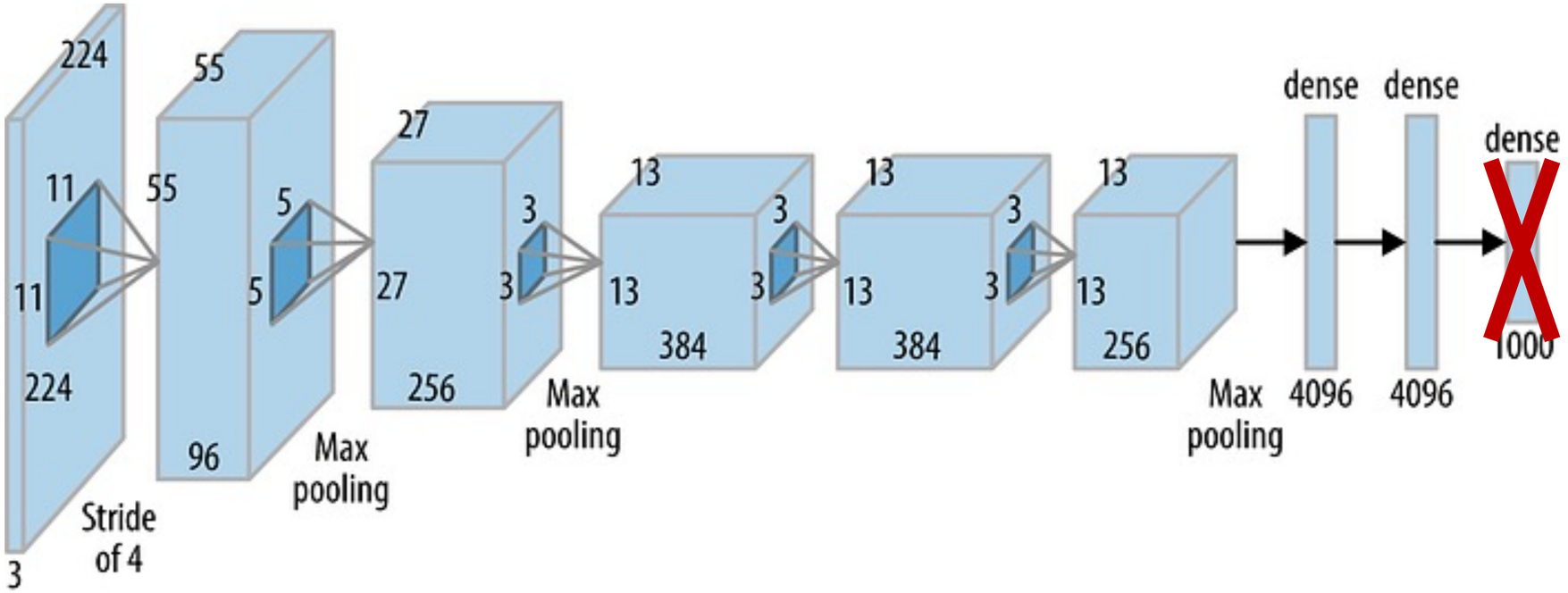
# Region-based Convolutional Network (R-CNN)

**ขั้นตอนที่ 2** นำ Region Proposal ที่ได้รับมาผ่านกระบวนการ Wrapping เพื่อให้รูปภาพอยู่ในขนาดมาตรฐานเดียวกันแบบจัตุรัสก่อน (224 x 224 พิกเซล) เนื่องจาก ..... ..... ผลลัพธ์ที่ได้จะเรียกว่า Wrapping Region



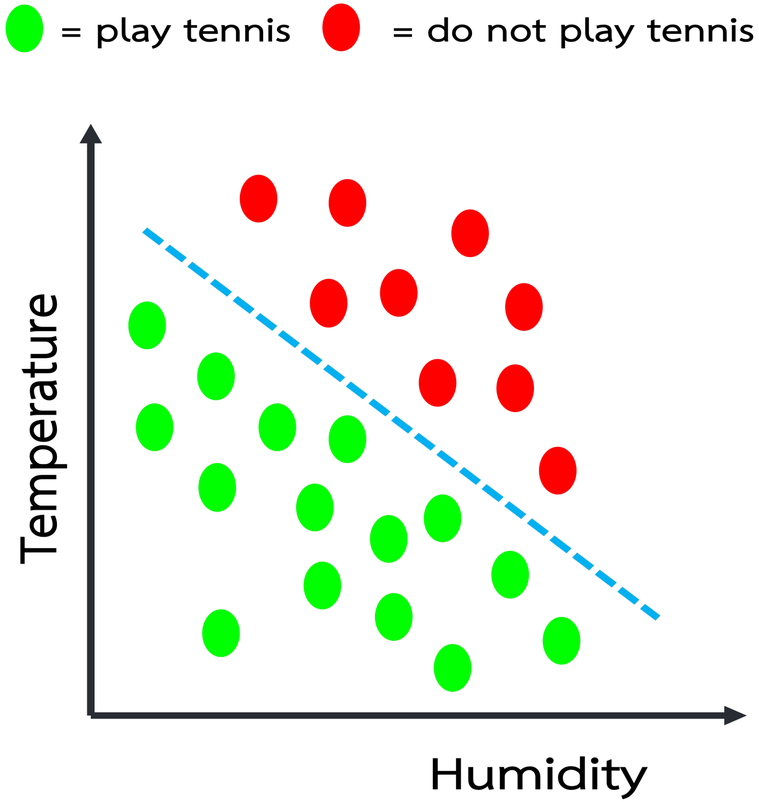
# Region-based Convolutional Network (R-CNN)

**ขั้นตอนที่ 3** นำ Wrapping Region มาสกัดคุณลักษณะด้วย Convolutional Neural Network (CNN) โดยอาศัยสถาปัตยกรรมแบบ AlexNet แต่จะตัด Fully Connected Layer สุดท้ายทิ้ง และผลลัพธ์ที่ได้จะเรียกว่า Feature Vector



# Region-based Convolutional Network (R-CNN)

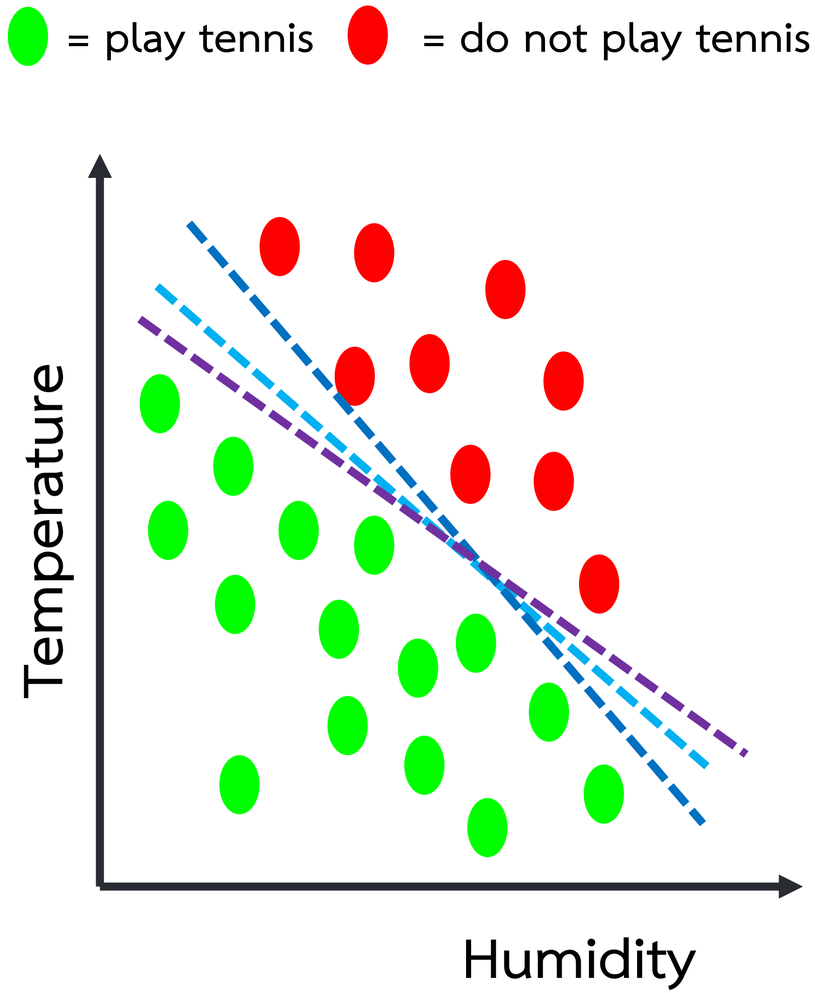
**ขั้นตอนที่ 4** นำ Feature Vector ส่งต่อไปยัง Support Vector Machine (SVM) เพื่อจำแนกว่าเป็นคลาสใด




**กำหนดให้**

- D คือ ข้อมูลตัวอย่าง (Sample data) โดยที่ $D = \{(x_i,y_i), \ldots, (x_i,y_i)\}$
- $x_i$ คือ แอททริบิวของแต่ละข้อมูล โดยที่ $x_i \in R^d$
- $y_i$ คือ กลุ่มหรือคลาสของข้อมูล $x_i$ โดยที่ $y_i \in \{-1, +1\}$

# Region-based Convolutional Network (R-CNN)

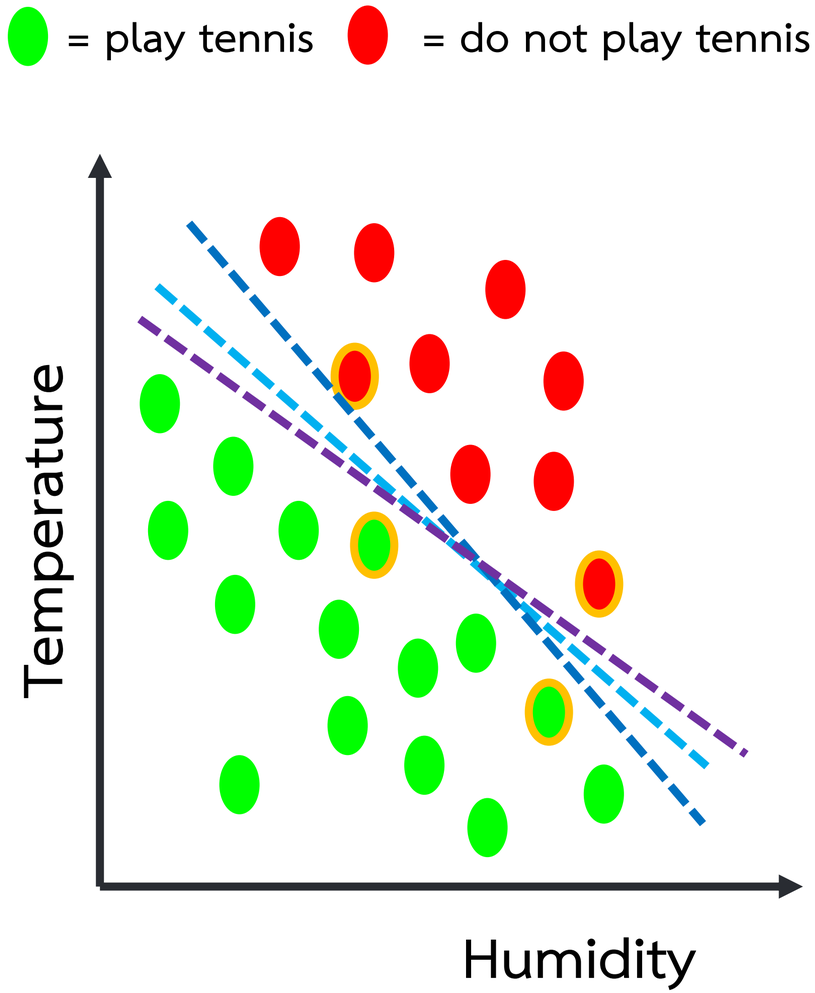



**กำหนดให้**

- D คือ ข้อมูลตัวอย่าง (Sample data) โดยที่ $D = \{(x_i,y_i), \ldots, (x_i,y_i)\}$
- $x_i$ คือ แอททริบิวของแต่ละข้อมูล โดยที่ $x_i \in R^d$
- $y_i$ คือ กลุ่มหรือคลาสของข้อมูล $x_i$ โดยที่ $y_i \in \{-1, +1\}$

จากภาพจะพบว่าระนาบเกิน (hyperplanes) ของข้อมูลชุดนี้มีจำนวนมาก ดังนั้น ระนาบเกินที่มีระยะห่างระหว่างเวกเตอร์สนับสนุนของข้อมูลทั้งสองกลุ่มมากที่สุดจะถูกเลือกมาใช้งาน

# Region-based Convolutional Network (R-CNN)

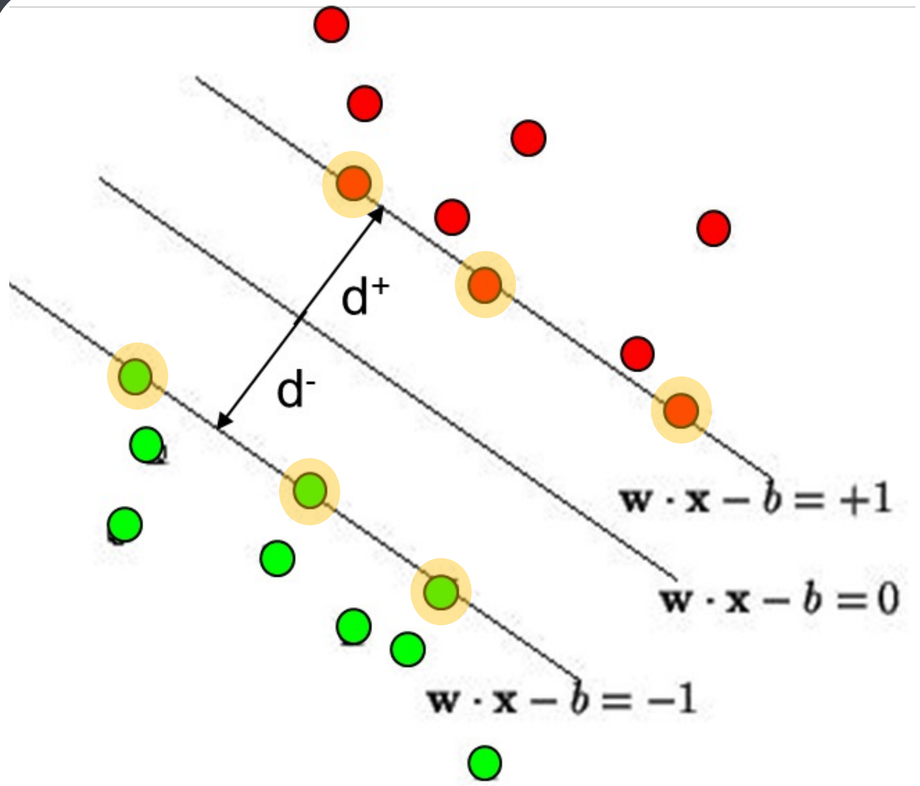



**กำหนดให้**

- D คือ ข้อมูลตัวอย่าง (Sample data) โดยที่ $D = \{(x_i,y_i), \ldots, (x_i,y_i)\}$
- $x_i$ คือ แอททริบิวของแต่ละข้อมูล โดยที่ $x_i \in R^d$
- $y_i$ คือ กลุ่มหรือคลาสของข้อมูล $x_i$ โดยที่ $y_i \in \{-1, +1\}$

จากภาพจะพบว่าระนาบเกิน (hyperplanes) ของข้อมูลชุดนี้มีจำนวนมาก ดังนั้น ระนาบเกินที่มีระยะห่างระหว่าง***เวกเตอร์สนับสนุน***ของข้อมูลทั้งสองกลุ่มมากที่สุดจะถูกเลือกมาใช้งาน

# Region-based Convolutional Network (R-CNN)




สามารถคำนวณหาระนาบเกินได้จากสมการเส้นตรงต่อไปนี้

$$WX + b = 0$$

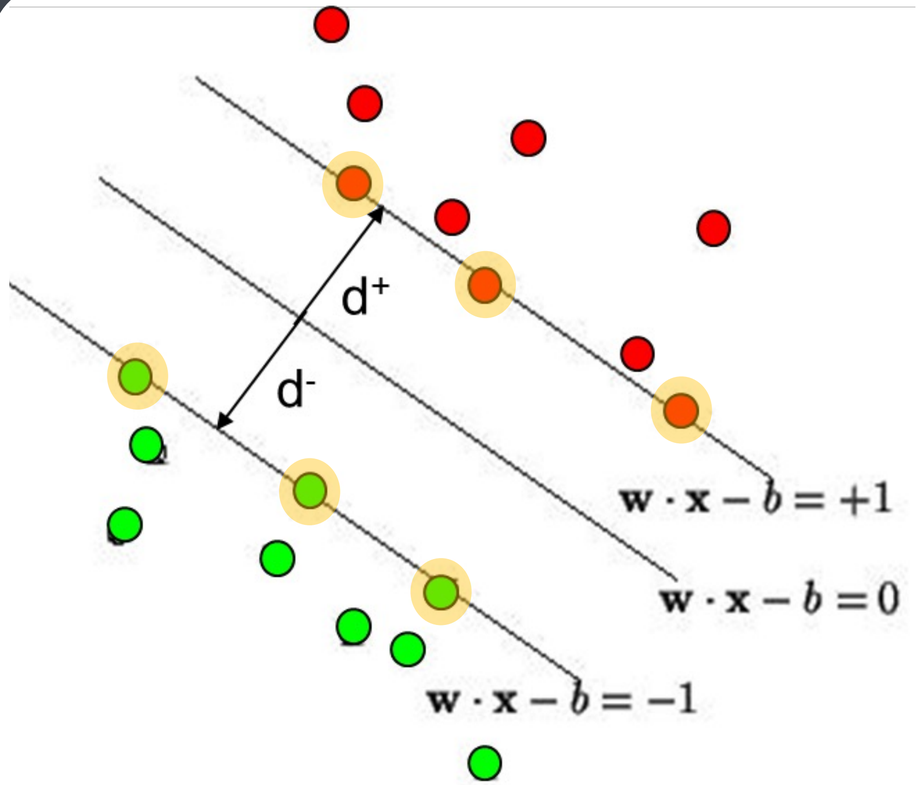
โดยที่ x คือ แอททริบิวของแต่ละข้อมูล W คือ ตัวแปรควบคุมระนาบเกิน และ b คือ ค่าคงที่

กำหนดให้

- H1 คือ ระนาบขอบเขตของข้อมูลกลุ่ม 1 ซึ่งสามารถเขียนสมการกำกับได้ดังนี้ $WX_i + b \geq +1$ ก็ต่อเมื่อ $Y_i$ เท่ากับ +1
- H2 คือ ระนาบขอบเขตของข้อมูลกลุ่ม 2 ซึ่งสามารถเขียนสมการกำกับได้ดังนี้ $WX_i + b \leq -1$ ก็ต่อเมื่อ $Y_i$ เท่ากับ -1

โดยที่ ข้อมูลที่อยู่บนระนาบขอบเขตจะเรียกว่า เวกเตอร์สนับสนุน (Support Vector)

# Region-based Convolutional Network (R-CNN)

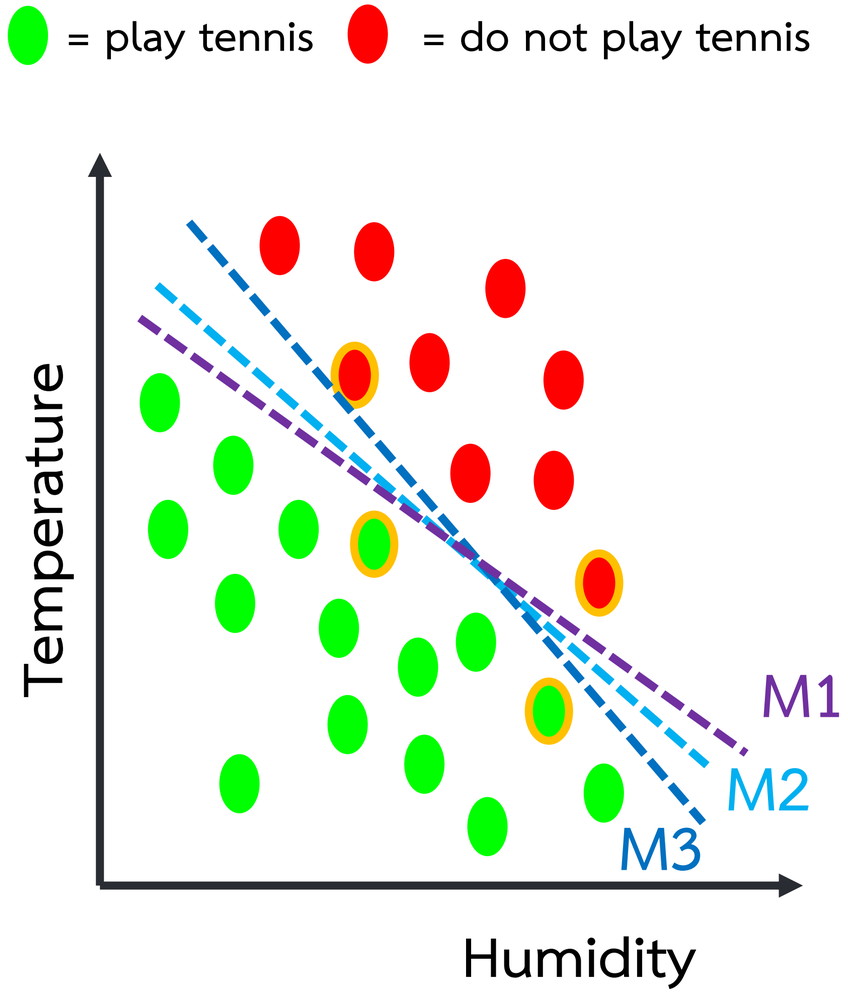



การคำนวณหาระยะห่างระหว่าง H1 และ H2 จะคำนวณได้จากสมการต่อไปนี้

$$M = d^{+} + d^{-}$$

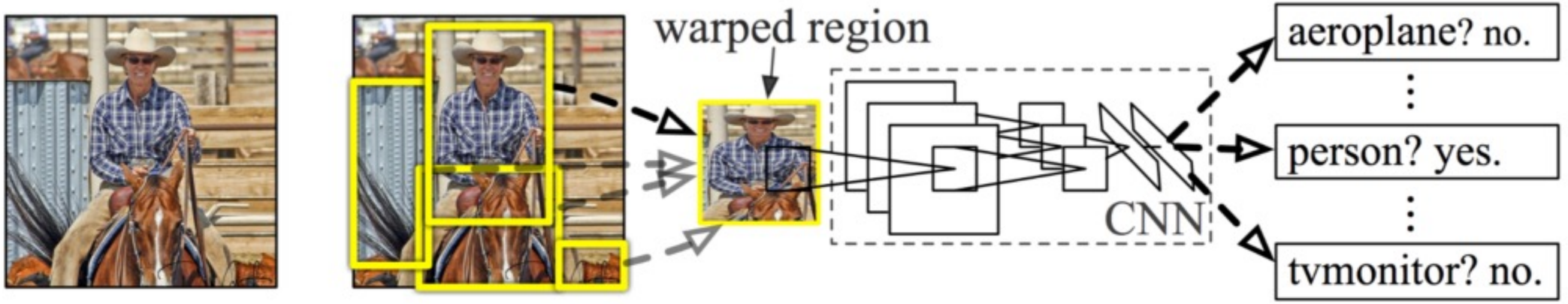
โดยที่ $d^{+}$ คือ ระยะทางที่สั้นที่สุดจาก H1 ถึงระนาบเกิน ขณะที่ $d^{-}$ คือ ระยะทางที่สั้นที่สุดจาก H2 ถึงระนาบเกิน

# Region-based Convolutional Network (R-CNN)




จากนั้น เราจะกำหนดระนาบส่วนเกินสำหรับข้อมูลชุดนี้โดยพิจารณาจากระนาบส่วนเกินที่มีค่า M มากที่สุด

# Region-based Convolutional Network (R-CNN)



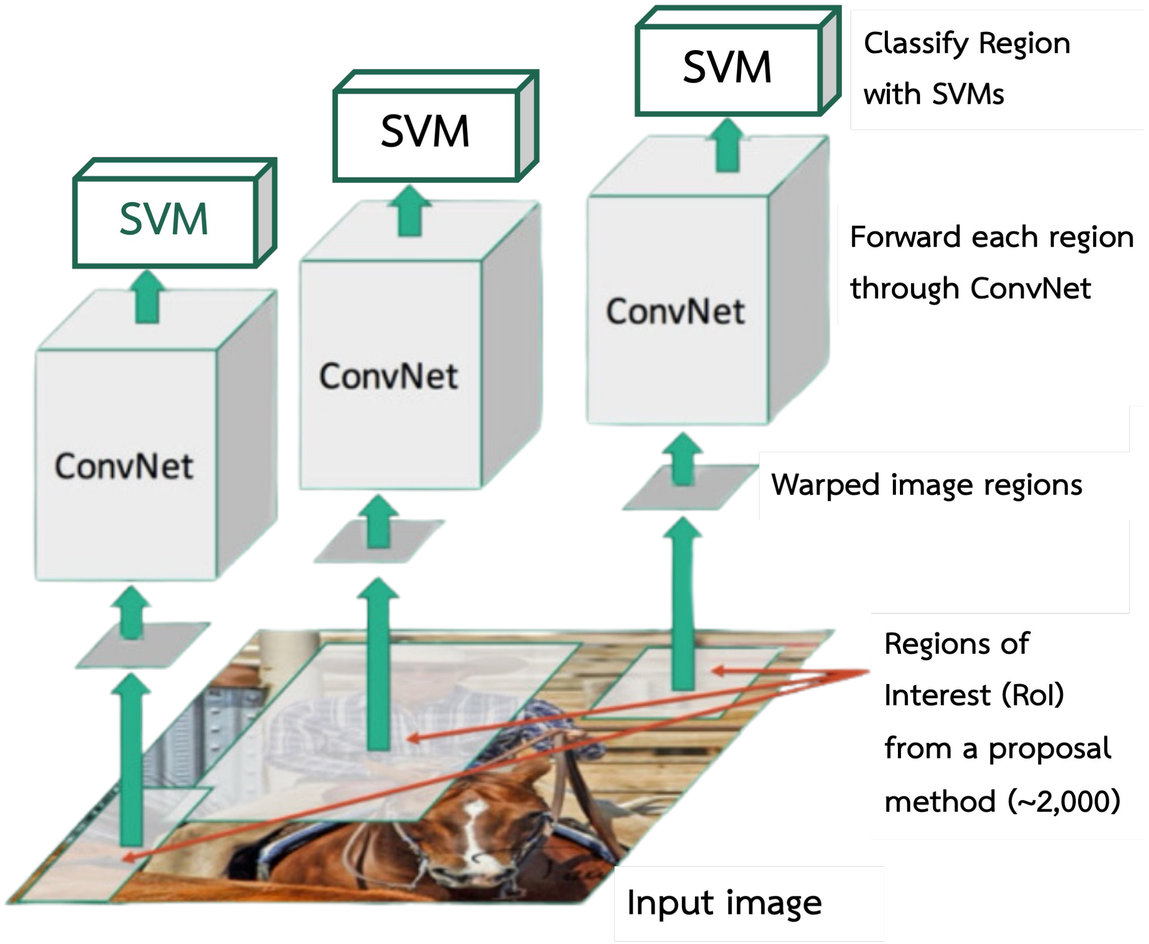

**1**. Input image

**2**. Extract region proposals (~2k)

**3**. Compute CNN features

**4**. Classify regions

# Region-based Convolutional Network (R-CNN)




(a) R-CNN

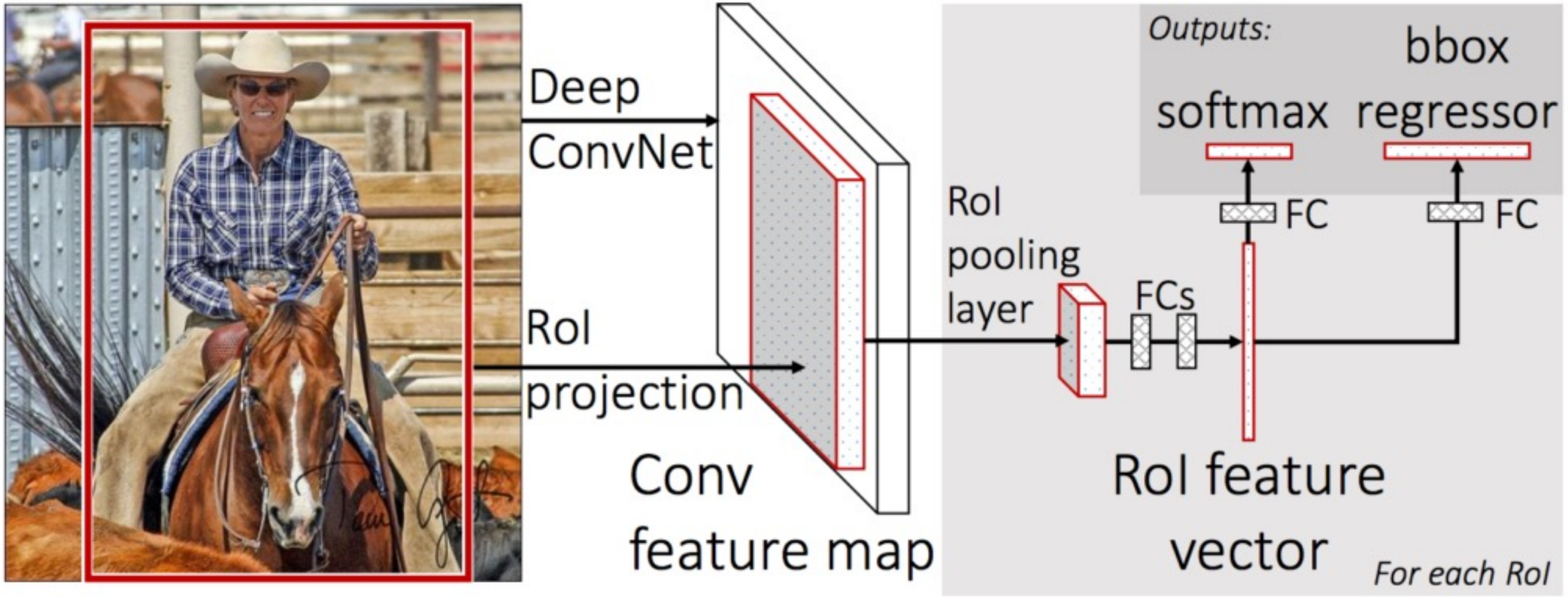
อ้างอิง https://analyticsindiamag.com/r-cnn-vs-fast-r-cnn-vs-faster-r-cnn-a-comparative-guide/



# หัวข้อ

- Multi-Stage Detectors
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- Single-Stage Detectors
  - You Only Look Once (YOLO)
- การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics

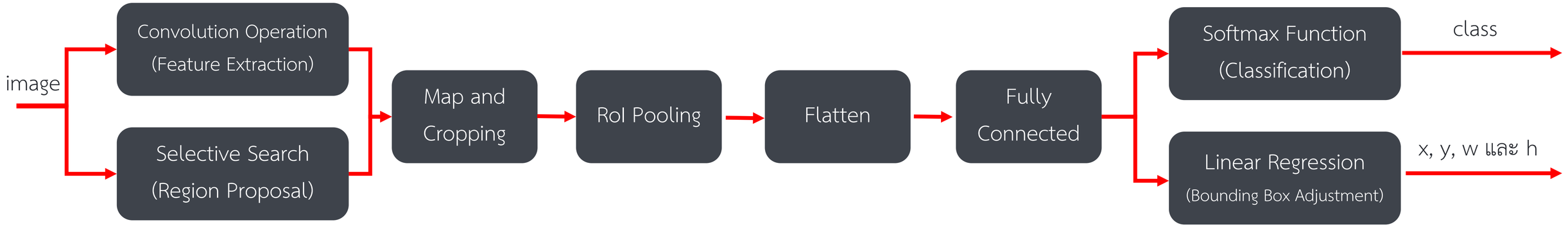
# Fast Region-based Convolutional Network (Fast R-CNN)




หลังจากนั้น Ross Girshick และคณะได้พัฒนา Fast R-CNN เพื่อลดเวลาที่ใช้ในการคำนวณลง ซึ่งมีขั้นตอนการทำงานคล้ายกับวิธีการ R-CNN

อ้างอิง https://towardsdatascience.com/r-cnn-fast-r-cnn-faster-r-cnn-yolo-object-detection-algorithms-36d53571365e

# Fast Region-based Convolutional Network (Fast R-CNN)




วิธีการ Fast R-CNN ประกอบไปด้วย 8 ขั้นตอนการทำงาน

อ้างอิง https://analyticsindiamag.com/r-cnn-vs-fast-r-cnn-vs-faster-r-cnn-a-comparative-guide/



# Fast Region-based Convolutional Network (Fast R-CNN)

**ขั้นตอนที่ 1** นำเข้าทั้งภาพ (Whole Image) เข้าสู่ CNN ส่วน Feature Extraction เท่านั้น เพื่อสกัด Feature Map จากทั้งภาพ แทนที่จะส่งเข้าทีละ Region Proposal สิ่งนี้ ทำให้วิธีการ Fast R-CNN จะเรียกใช้ Convolution Operation 1 ครั้งต่อ 1 ภาพ เท่านั้น ผลลัพธ์ที่ได้ คือ Feature Map

**ขั้นตอนที่ 2** ตรวจจับวัตถุในภาพด้วย Selective Search จากทั้งภาพ (Whole Image) ผลลัพธ์ที่ได้ คือ Region Proposal

**ขั้นตอนที่ 3** นำตำแหน่งของแต่ละ Region Proposal ได้แก่ (x, y) width และ height ไปทาบลง Feature Map และ Cropping บริเวณดังกล่าว เพื่อให้ได้รับ Feature Map ของแต่ละ Region Proposal ที่มีขนาดไม่เท่ากัน ด้วยหลักการ ROI projection

**ขั้นตอนที่ 4** ปรับขนาดและรูปร่างของ Feature Map ให้อยู่ในขนาดเดียวกันแบบจัตุรัส ด้วยหลักการ RoI Pooling

**ขั้นตอนที่ 5** ปรับรูปแบบ Feature Map ของแต่ละ Region Proposal ให้อยู่ในรูปแบบ Feature Vector ก่อนส่งต่อเข้าสู่ Fully Connected Layer

**ขั้นตอนที่ 6** ส่ง Feature Vector เข้าสู่ Fully Connected Layer

**ขั้นตอนที่ 7** จำแนกคลาสของ Feature Vector ในแต่ละ Region Proposal ด้วย Softmax Function

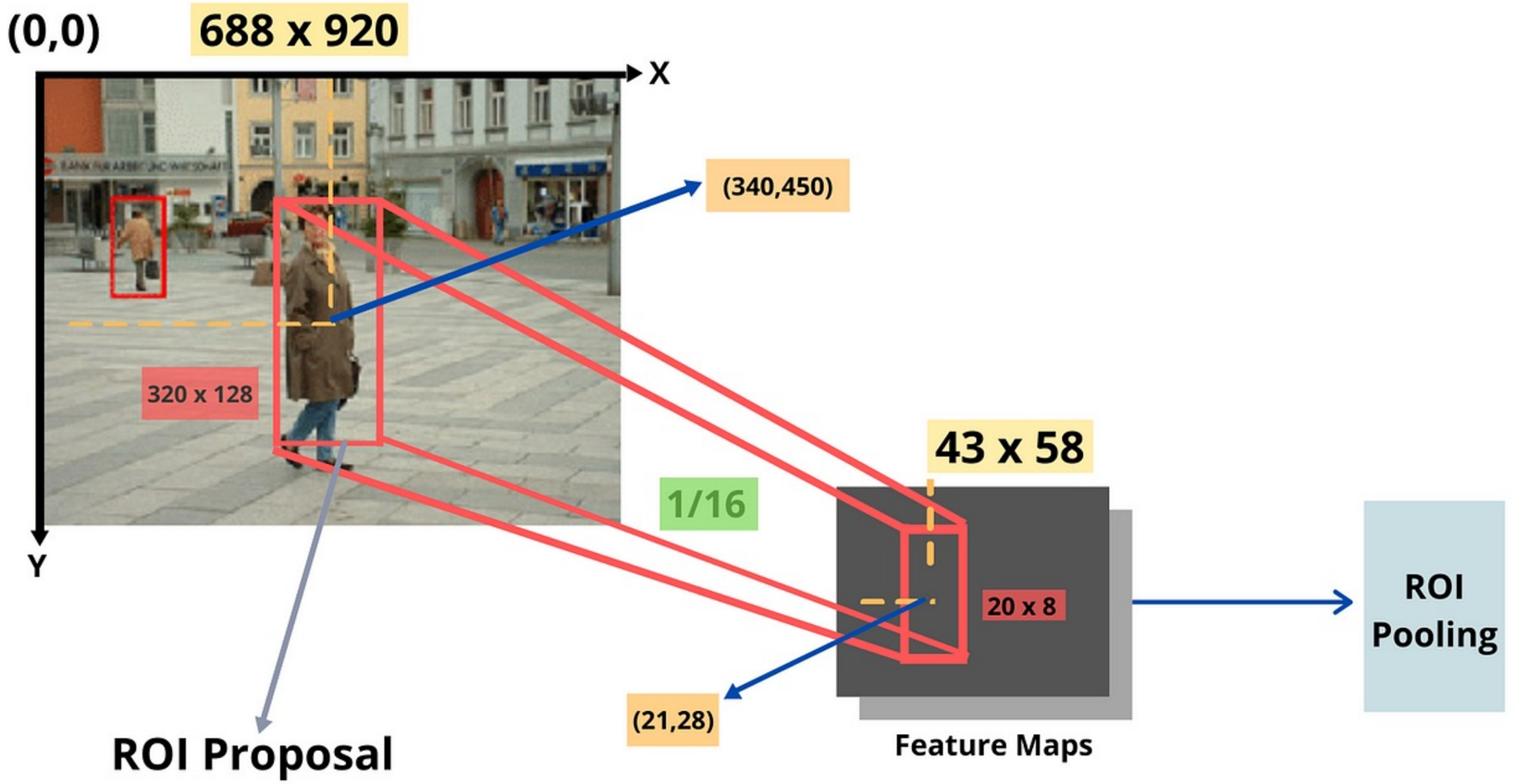
**ขั้นตอนที่ 8** ปรับเปลี่ยนตำแหน่งของ Region Proposal ให้สอดคล้องกับวัตถุขึ้นด้วย Linear Regression จาก x, y, w และ h

อางอิง https://analyticsindiamag.com/r-cnn-vs-fast-r-cnn-vs-faster-r-cnn-a-comparative-guide/

# Fast Region-based Convolutional Network (Fast R-CNN)

## หลักการ ROI projection

ผู้ใช้งานจะได้รับตำแหน่ง (x,yw,h) ของแต่ละ Region Proposal จากการทำ Selective Search จากนั้นต้องการนำคำนวณหาตำแหน่งบน Feature Map โดยอาศัยการคำนวณเทียบตำแหน่งและสัดส่วน นี้คือเหตุผลหลักว่าทำไมวิธีการ Fast R-CNN ถึงจำเป็นต้องกำหนดขนาดภาพนำเข้าแบบตายตัว




อ้างอิง https://towardsdatascience.com/understanding-fast-r-cnn-and-faster-r-cnn-for-object-detection-adbb55653d97

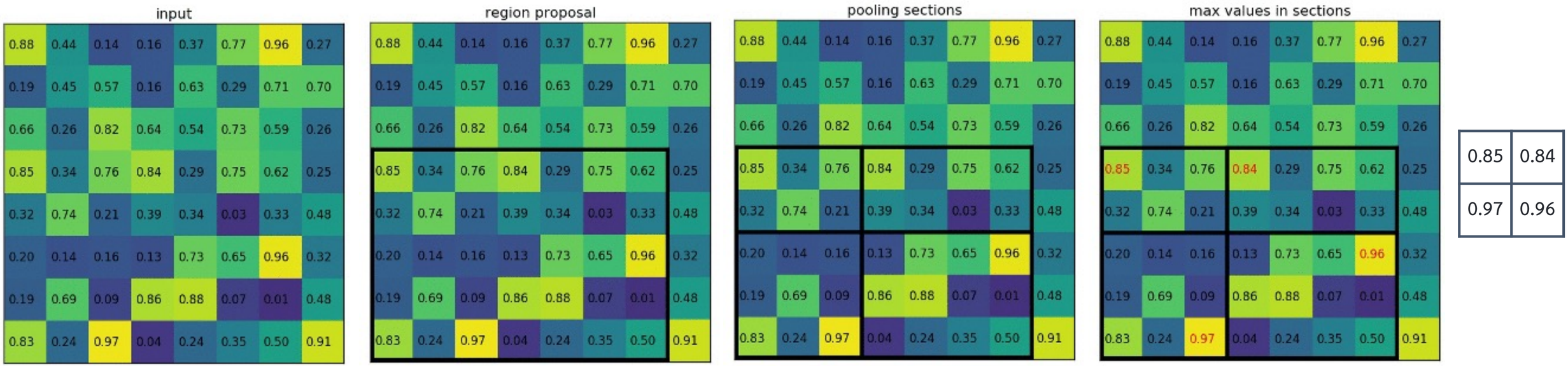
# Fast Region-based Convolutional Network (Fast R-CNN)

**ขั้นตอนการทำงานของ ROI Pooling ประกอบด้วย 3 ขั้นตอน**

**ขั้นตอนที่ 4.1** นำตำแหน่งของแต่ละ Region Proposal ไปทาบลง Feature Map และ Cropping บริเวณดังกล่าว เพื่อให้ได้รับ Feature Map ของแต่ละ Region Proposal ที่มีขนาดไม่เท่ากัน

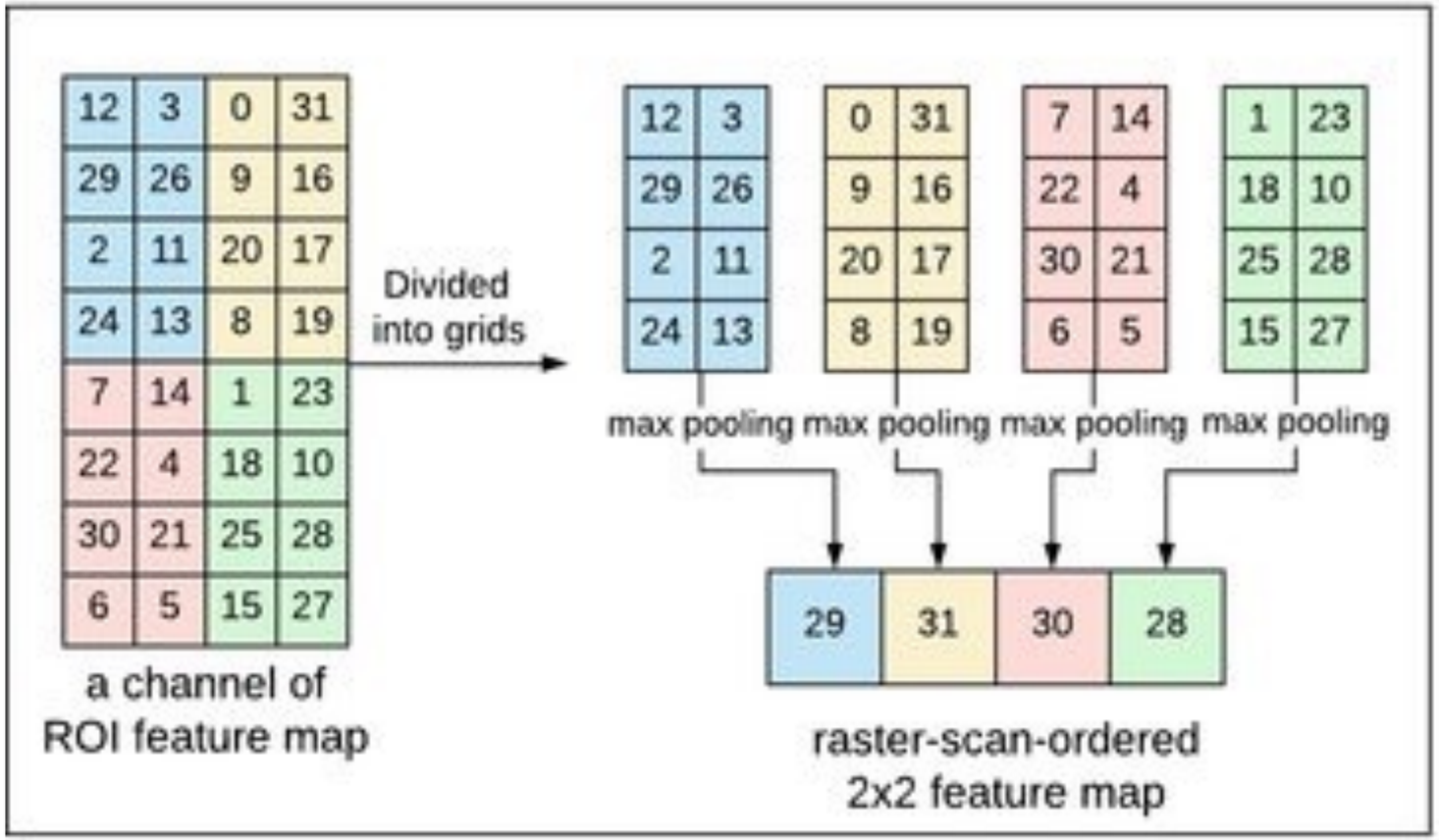
**ขั้นตอนที่ 4.2** แบ่ง Feature Map บริเวณดังกล่าวออกเป็นส่วน ด้วยการคำนวณจาก $\left\lfloor \frac{m}{n} \right\rfloor$ โดยที่ m ความสูงหรือความกว้างและ n คือ จำนวนชิ้นส่วนที่ต้องการแบ่ง

**ขั้นตอนที่ 4.3** หาค่าสูงสุดในแต่ละส่วน (Max Pooling)




https://towardsdatascience.com/region-of-interest-pooling-f7c637f409af

# Fast Region-based Convolutional Network (Fast R-CNN)

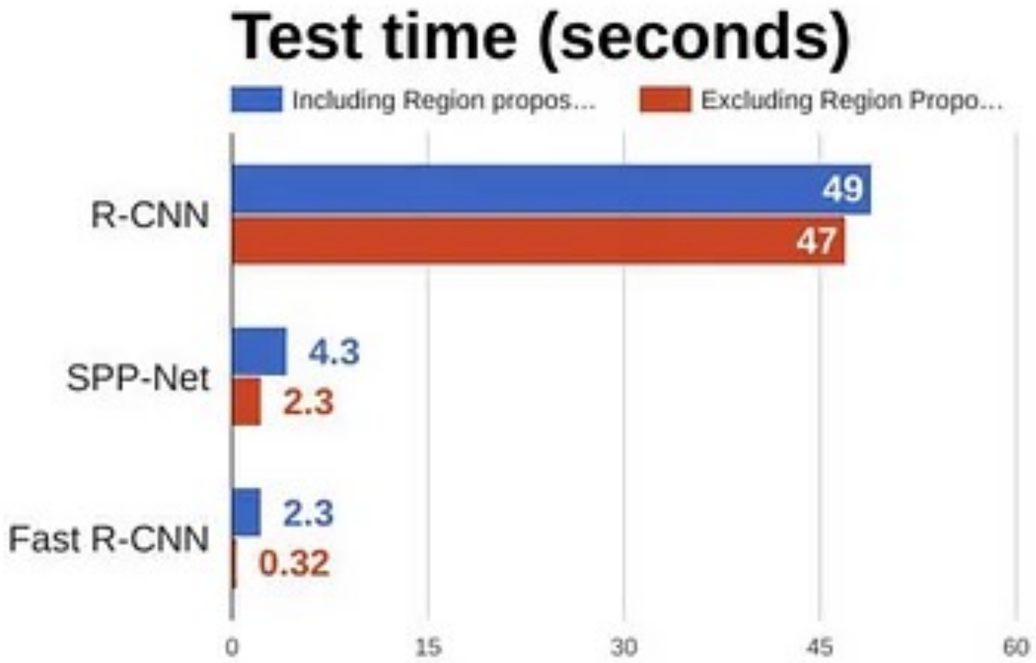



Fan, Xiaochuan & Guo, Hao & Zheng, Kang & Feng, Wei & Wang, Song. (2018). Object Detection with Mask-based Feature Encoding.

# Fast Region-based Convolutional Network (Fast R-CNN)

เหตุผลที่วิธีการ Fast R-CNN ทำงานได้รวดเร็วกว่าวิธีการ R-CNN เนื่องจากไม่ต้องสกัด Feature Map ที่ละ Region Proposal แต่วิธีการ Fast R-CNN จะเรียกใช้ Convolution Operation 1 ครั้งต่อ 1 ภาพ



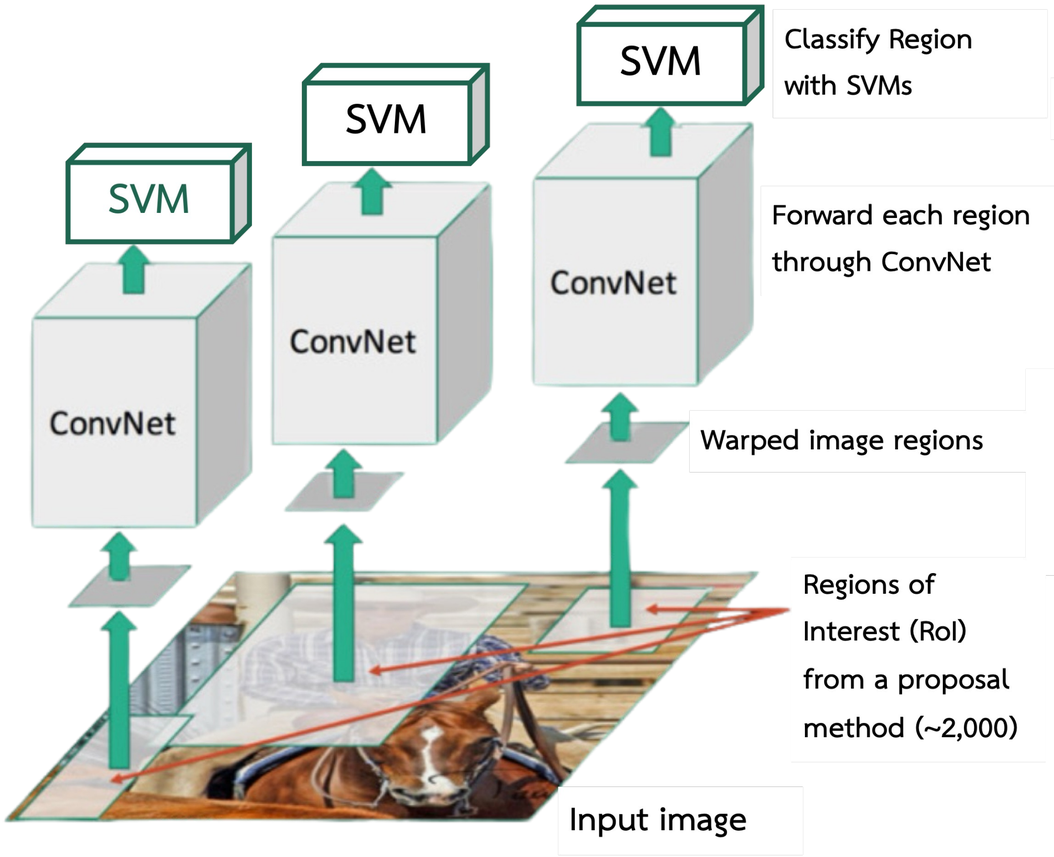

อ้างอิง https://towardsdatascience.com/r-cnn-fast-r-cnn-faster-r-cnn-yolo-object-detection-algorithms-36d53571365e

# Fast Region-based Convolutional Network (Fast R-CNN)

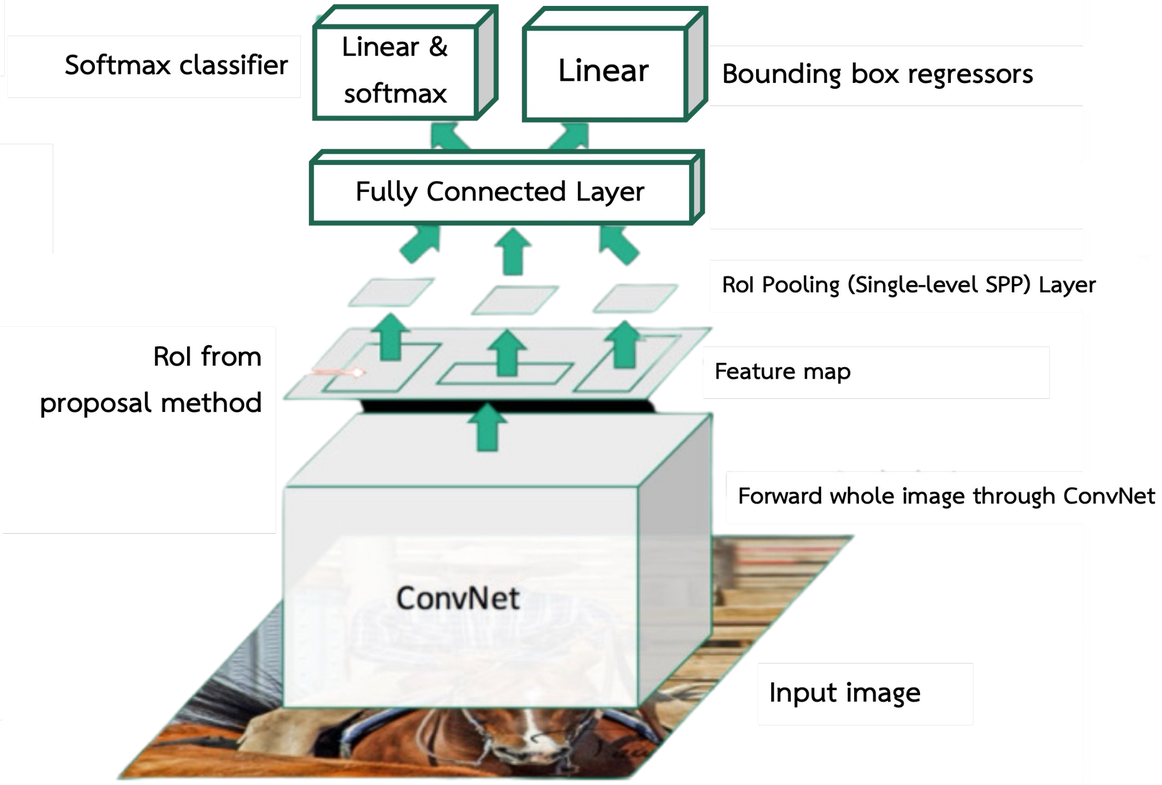
| | RCNN | Fast RCNN |
|---|---|---|
| Training Time | 84 hours | 9.5 hours |
| (Speedup) | 1x | 8.8x |
| Test time per image | 47 seconds | 0.32 seconds |
| (Speedup) | 1x | 146x |
| Test time per image with Selective Search | 50 seconds | 2 seconds |
| (Speedup) | 1x | 25x |

อ้างอิง https://towardsdatascience.com/understanding-fast-r-cnn-and-faster-r-cnn-for-object-detection-adbb55653d97

# Fast Region-based Convolutional Network (Fast R-CNN)




(a) R-CNN




(b) Fast R-CNN

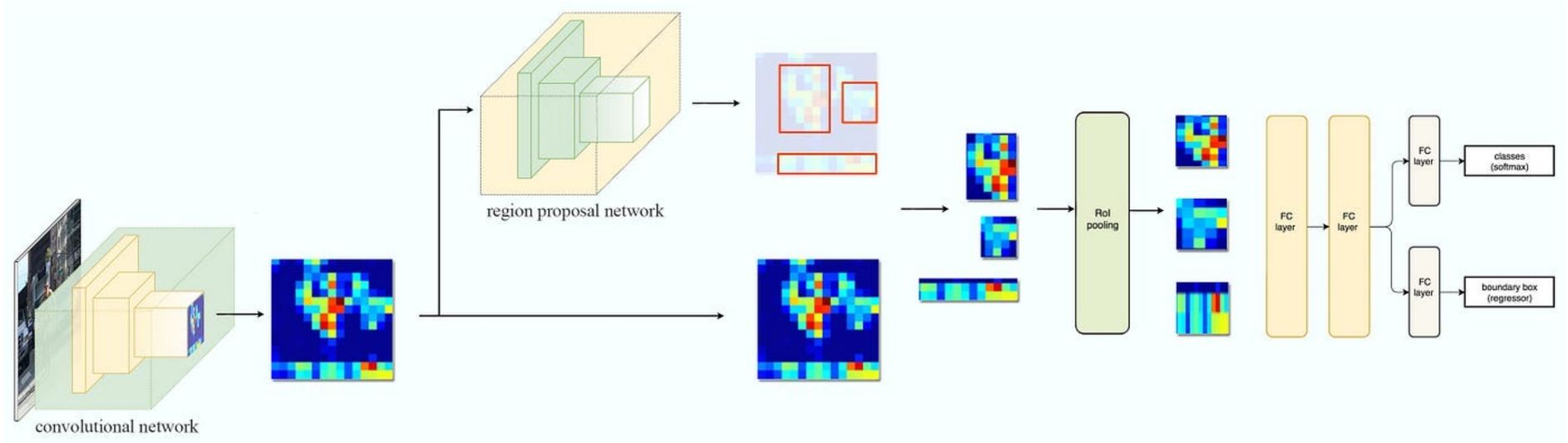
อ้างอิง https://towardsdatascience.com/r-cnn-fast-r-cnn-faster-r-cnn-yolo-object-detection-algorithms-36d53571365e



# หัวข้อ

- Multi-Stage Detectors
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- Single-Stage Detectors
  - You Only Look Once (YOLO)
- การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics

# Faster Region-based Convolutional Network (Faster R-CNN)




ในปี ค.ศ. 2015 ทีมนักวิจัย Shaoqing Ren และคณะได้พัฒนา Faster R-CNN ขึ้น เพื่อแก้ไขปัญหาการทำงานที่เชื่องช้าและใช้ทรัพยากรค่อนข้างมากของ Selective Search

อ้างอิง https://jonathan-hui.medium.com/image-segmentation-with-mask-r-cnn-ebe6d793272



# Faster Region-based Convolutional Network (Faster R-CNN)

**ขั้นตอนที่ 1** นำเข้าทั้งภาพ (Whole Image) เข้าสู่ CNN ส่วน Feature Extraction เท่านั้น เพื่อสกัด Feature Map จากทั้งภาพ แทนที่จะส่งเข้าทีละ Region Proposal สิ่งนี้ ทำให้วิธีการ Fast R-CNN จะเรียกใช้ Convolution Operation 1 ครั้งต่อ 1 ภาพ เท่านั้น ผลลัพธ์ที่ได้ คือ Feature Map

**ขั้นตอนที่ 2** ตรวจจับวัตถุในภาพด้วย Region Proposal Network (RPN) จากทั้งภาพ ผลลัพธ์ คือ Region Proposal

**ขั้นตอนที่ 3** นำตำแหน่งของแต่ละ Region Proposal ได้แก่ (x, y) width และ height ไปทาบลง Feature Map และ Cropping บริเวณดังกล่าว เพื่อให้ได้รับ Feature Map ของแต่ละ Region Proposal ที่มีขนาดไม่เท่ากัน

**ขั้นตอนที่ 4** ปรับขนาดและรูปร่างของ Feature Map ให้อยู่ในขนาดเดียวกันแบบจัตุรัส ด้วยหลักการ RoI Pooling

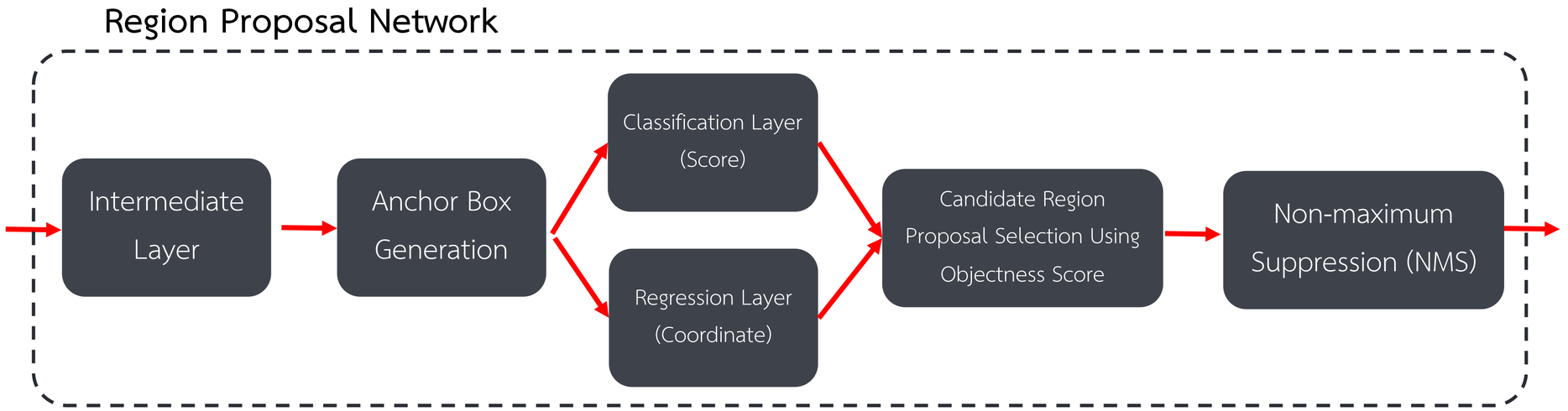
**ขั้นตอนที่ 5** ปรับรูปแบบ Feature Map ของแต่ละ Region Proposal ให้อยู่ในรูปแบบ Feature Vector ก่อนส่งต่อเข้าสู่ Fully Connected Layer

**ขั้นตอนที่ 6** ส่ง Feature Vector เข้าสู่ Fully Connected Layer

**ขั้นตอนที่ 7** จำแนกคลาสของ Feature Vector ในแต่ละ Region Proposal ด้วย Softmax Function

**ขั้นตอนที่ 8** ปรับเปลี่ยนตำแหน่งของ Region Proposal ให้สอดคล้องกับวัตถุขึ้นด้วย Linear Regression จาก x, y, w และ h

# Faster Region-based Convolutional Network (Faster R-CNN)

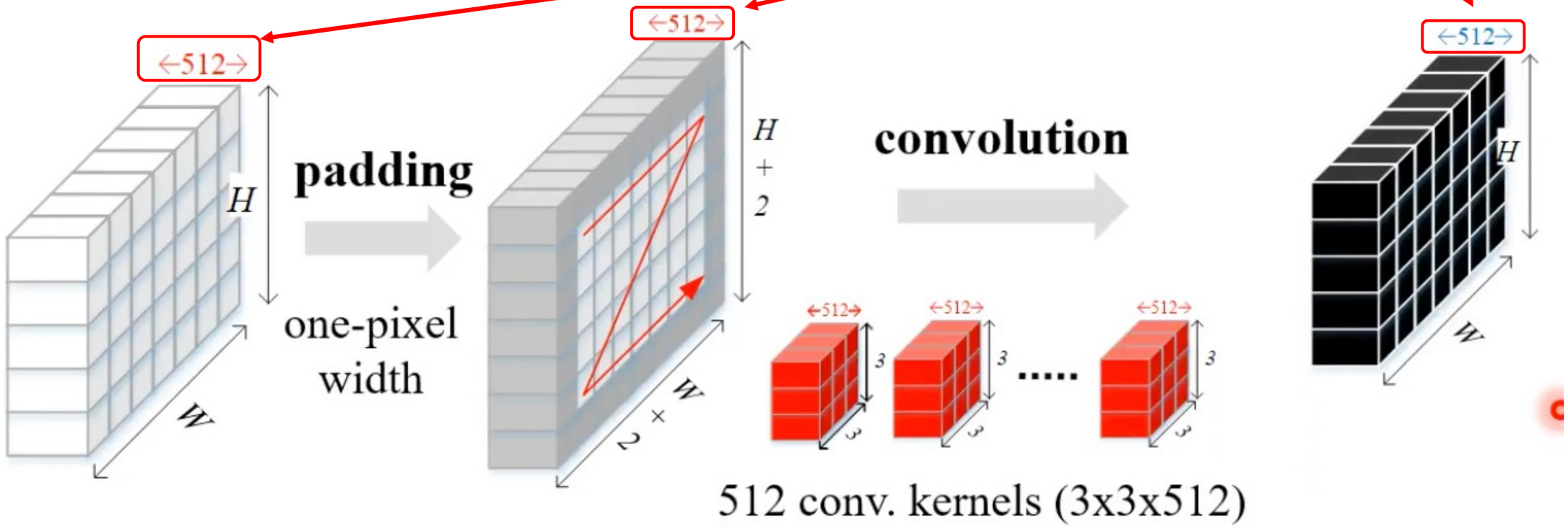



Region Proposal Network ประกอบไปด้วย 6 ขั้นตอน

# Faster Region-based Convolutional Network (Faster R-CNN)

Region Proposal Network ประกอบไปด้วย 6 ขั้นตอน

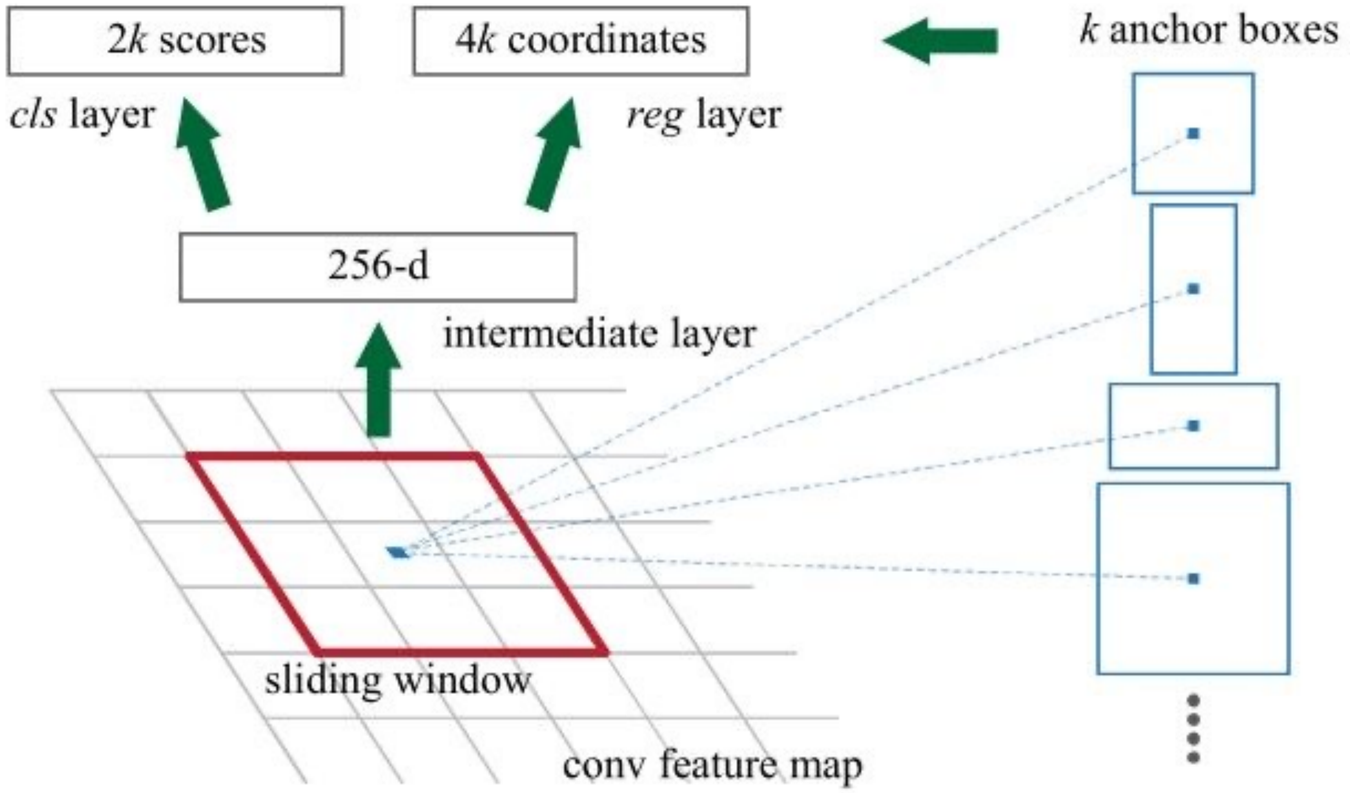
**ขั้นตอนที่ 2.1** นำ Feature Map ของ Shared Convolution Layer สุดท้ายของ CNN มา สกัด Feature Map โดยอาศัย Padding และ 3x3 Convolution Window จำนวน 512 Filter **(จำนวน input และ output channel เท่ากัน)**




อ้างอิง https://www.youtube.com/watch?v=0tBhRfEzUWs

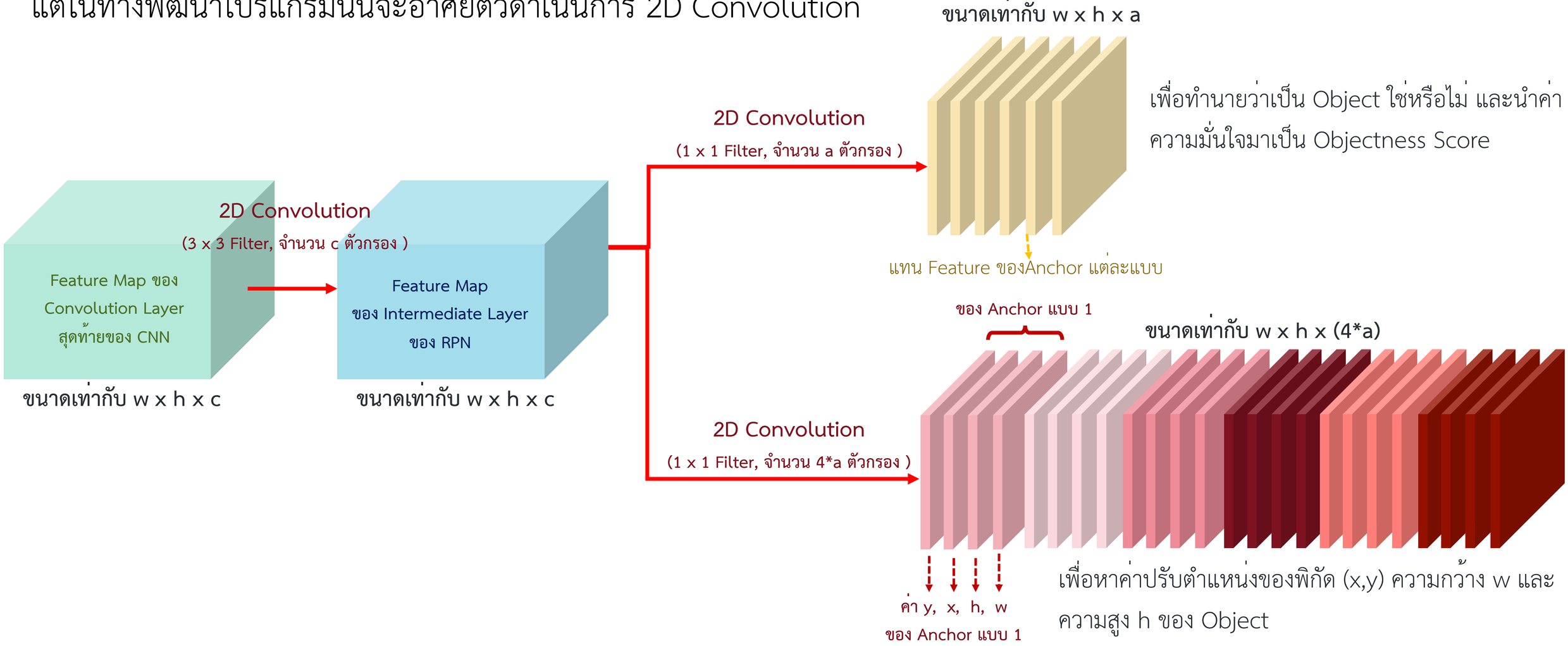
# Faster Region-based Convolutional Network (Faster R-CNN)

**ขั้นตอนที่ 2.2 ในทางทฤษฎี คือ เปรียบได้กับ**การสร้าง Anchor Boxes จากนำ Feature Map ของ Intermediate Layer สุดท้ายของ CNN มา Sliding Window แบบสี่เหลี่ยมขนาด N x M โดยกำหนดพารามิเตอร์ 2 ตัว ได้แก่ (1) ขนาด Scale และ (2) อัตราส่วน Aspect Ratio



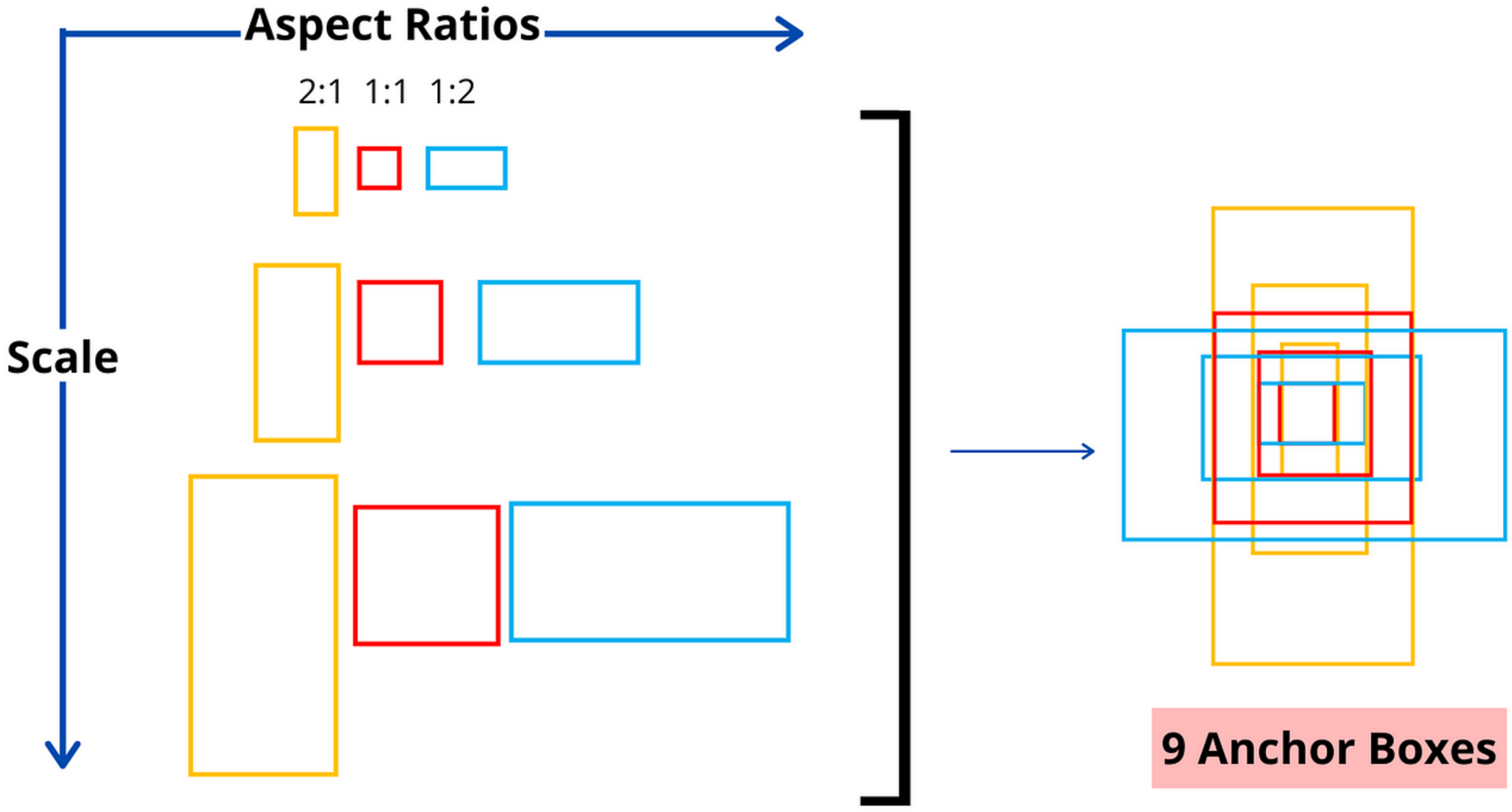
# Faster Region-based Convolutional Network (Faster R-CNN)

แต่ในทางพัฒนาโปรแกรมนั้นจะอาศัยตัวดำเนินการ 2D Convolution




หมายเหตุ a คือ จำนวนของรูปแบบ Anchors ทั้งหมด

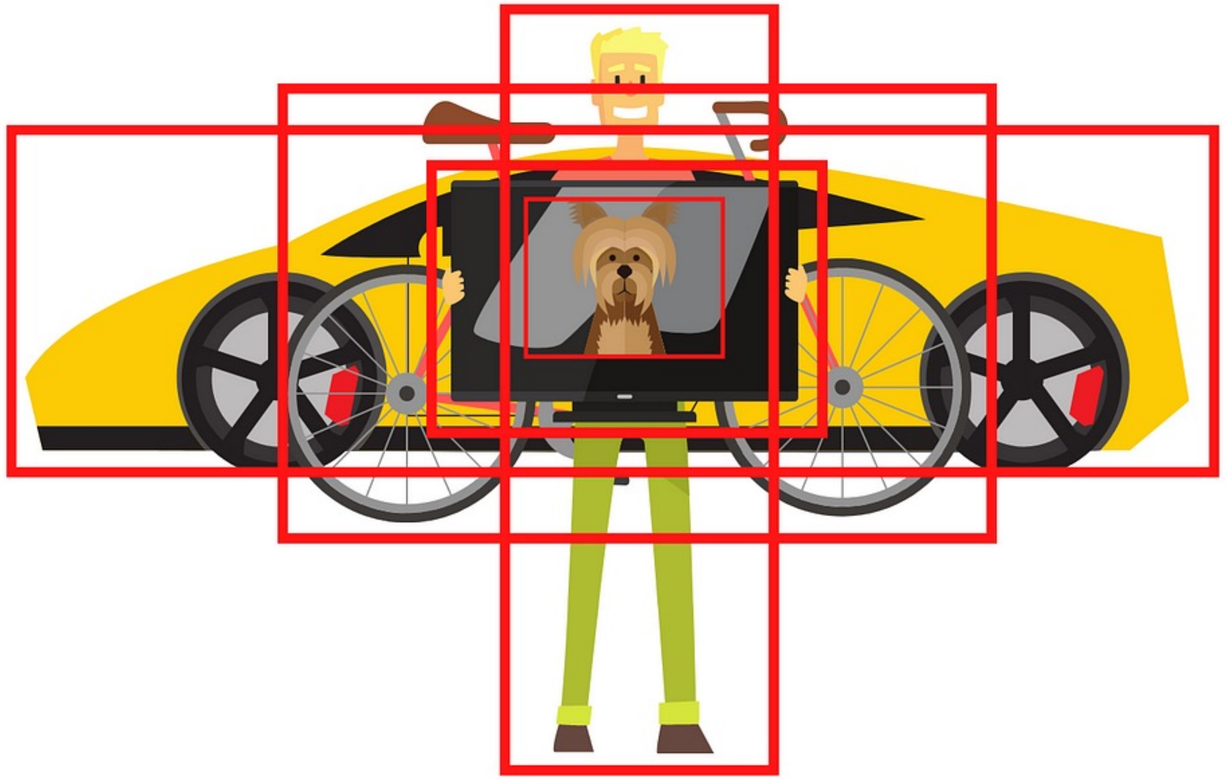
# Faster Region-based Convolutional Network (Faster R-CNN)




โดยปกติ จะกำหนดให Anchor Boxes มี 3 ขนาดและ 3 อัตราสวน ทำให 1 บริเวณมี 9 Anchor Boxes

อ้างอิง https://contohlengkap.github.io/ilustrasi/post/region-proposal-network/

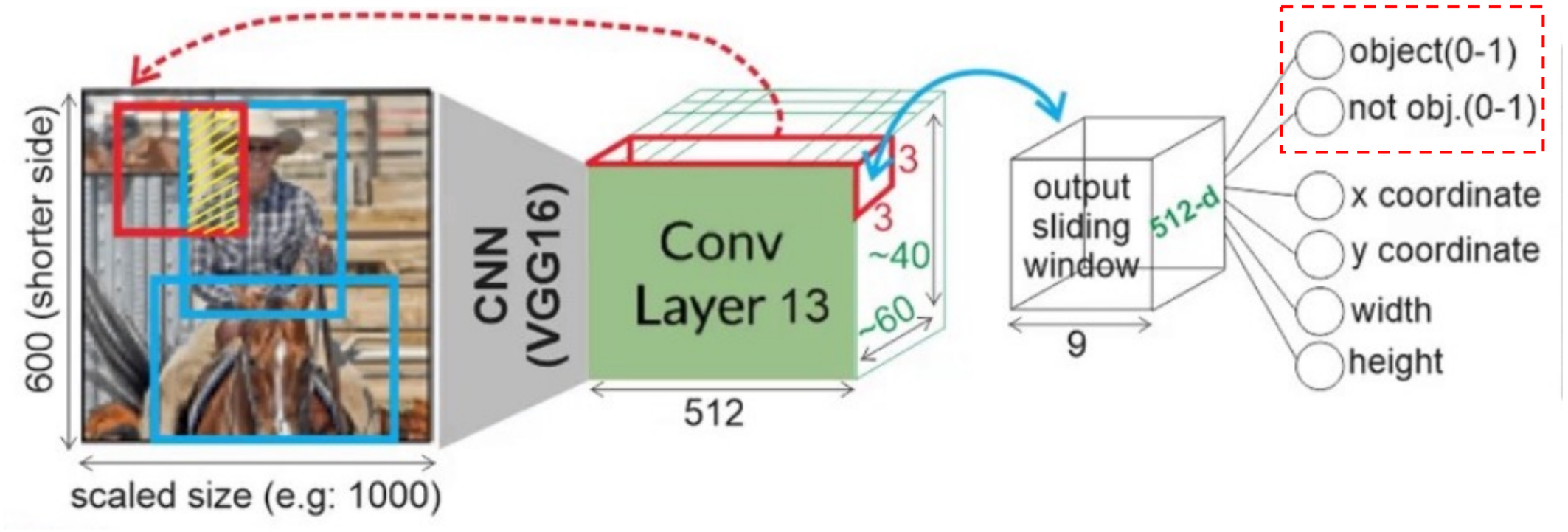
# Faster Region-based Convolutional Network (Faster R-CNN)



อ้างอิง https://towardsdatascience.com/understanding-fast-r-cnn-and-faster-r-cnn-for-object-detection-adbb55653d97

# Faster Region-based Convolutional Network (Faster R-CNN)

**ขั้นตอนที่ 2.3** การจำแนกว่า Anchor Boxes นั้นเป็น Foreground หรือ Background และมีค่า Objectiveness Score เท่าไหร่ โดยรับอินพุธเป็น Feature Vector และประมวลผลด้วย Fully Connected Layer

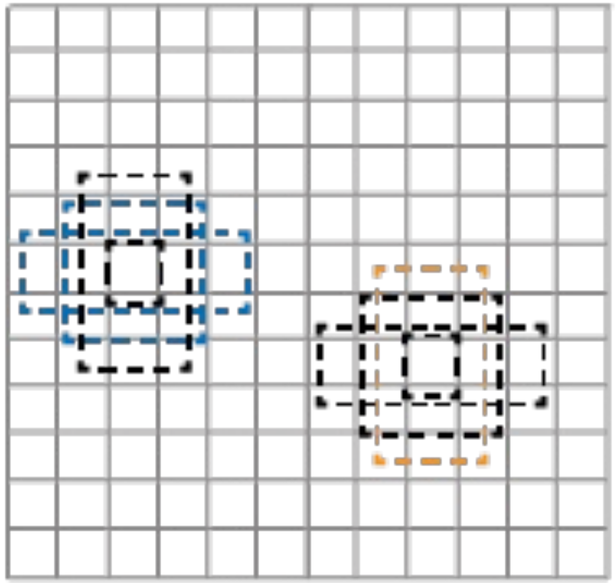


# Faster Region-based Convolutional Network (Faster R-CNN)

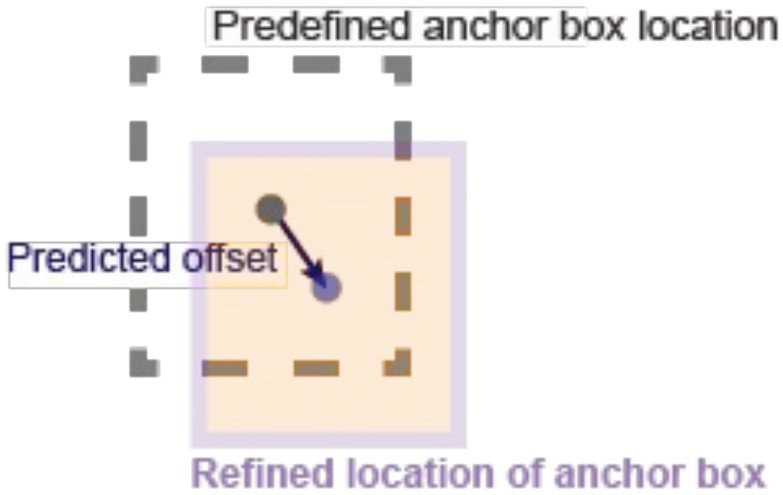
**ขั้นตอนที่ 2.4** การประมาณและปรับปรุงตำแหน่งของ Anchor Boxes โดยรับอินพุธเป็น [x, y, w, h] และประมวลผลด้วย Fully Connected Layer



Ground truth image and bounding boxes



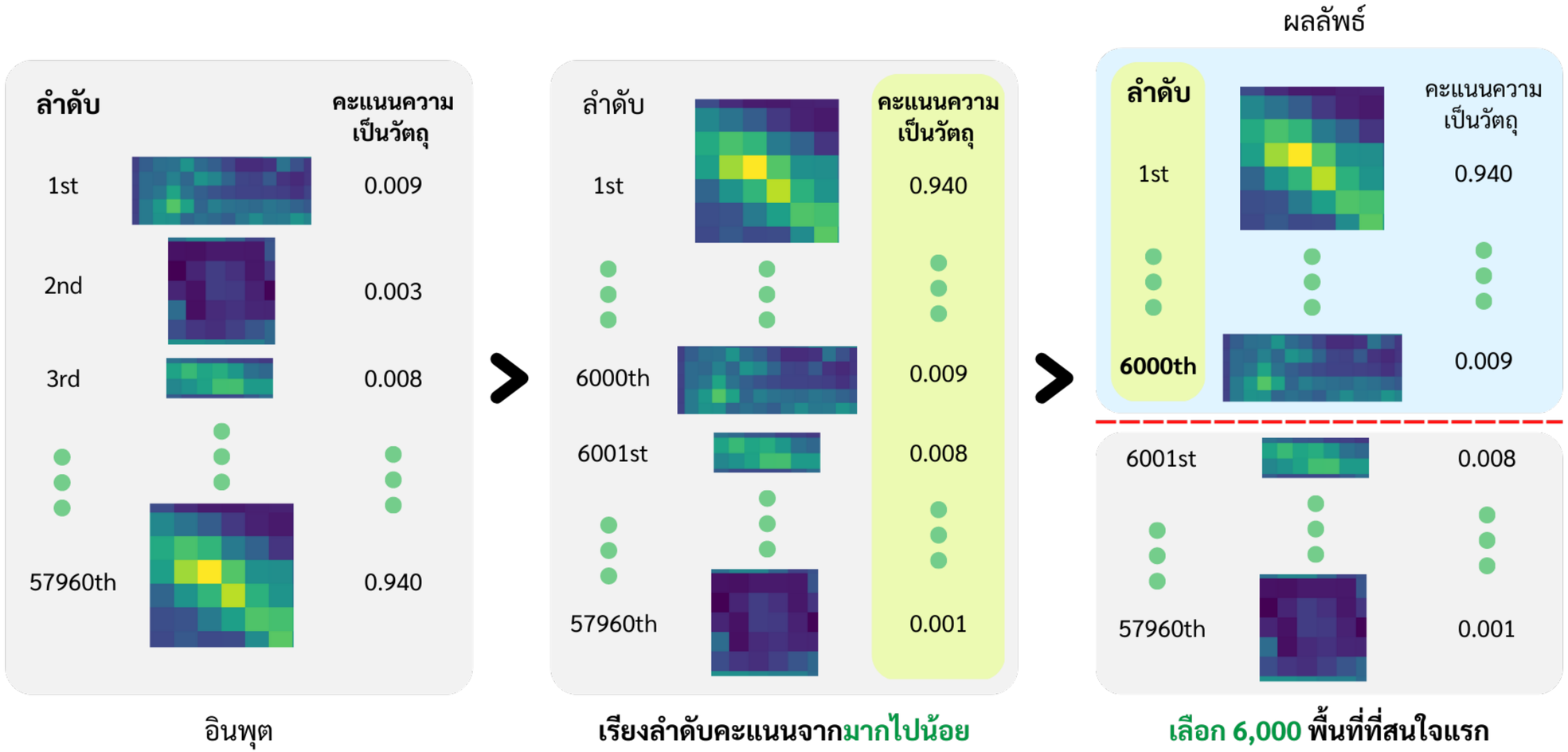
Anchor boxes at each predefined location in each feature map




อ้างอิง https://www.wovenware.com/blog/2020/06/anchor-boxes-in-object-detection-when-where-and-how-to-propose-them-for-deep-learning-apps/#.YVyIwBNKg8M

# Faster Region-based Convolutional Network (Faster R-CNN)

**ขั้นตอนที่ 2.5** นำ Candidate Region Proposal เฉพาะที่ได้รับการทำนายว่าเป็น Foreground มาเรียงตามค่า Objectness Score จากมากไปน้อย จากนั้น เลือกมา n ตัวแรกเท่านั้น เพื่อให้เป็น Region Proposal



# Faster Region-based Convolutional Network (Faster R-CNN)

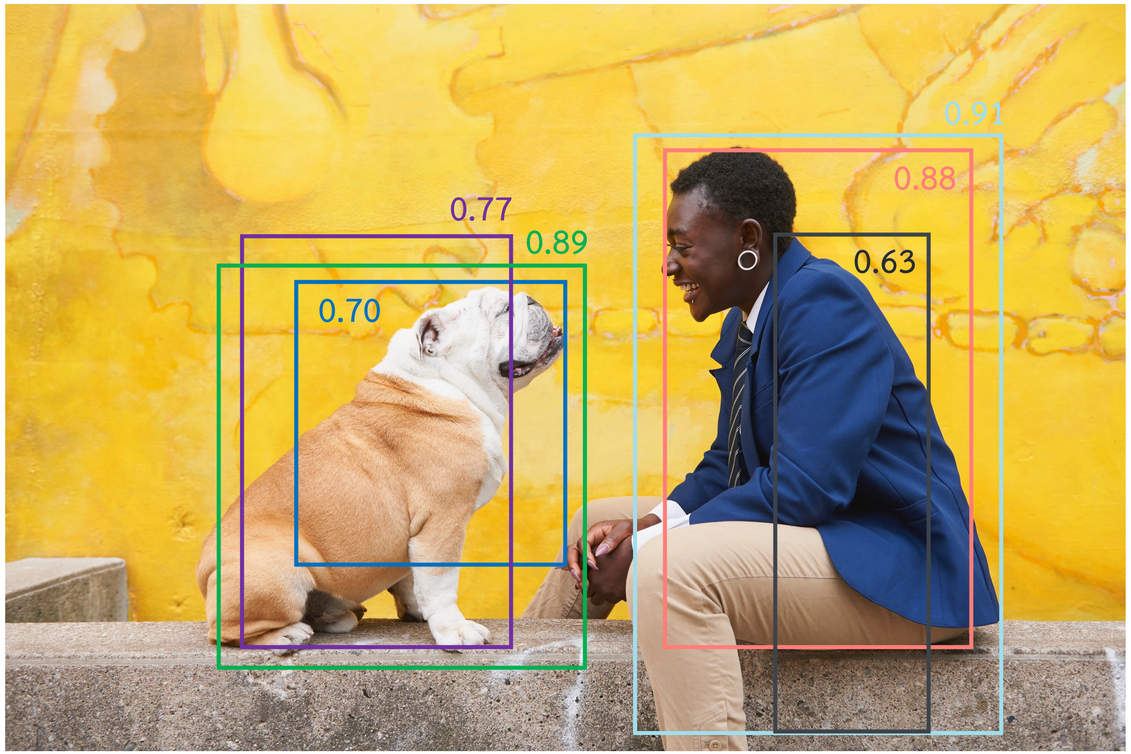
**ขั้นตอนที่ 2.6** นำ Region Proposal มาลดจำนวนลง ด้วยการพิจารณาความซ้ำเชิงพื้นที่จากค่า Objectness Score ด้วยวิธีการ Non-maximum Suppression (NMS) เพื่อให้ได้รับ Region Proposal ที่มีความเป็นไปได้สูงสุดมาใช้งานต่อ




อ้างอิง https://lengyi.medium.com/non-maximum-suppression-using-python-40be1df9c2a6

# Faster Region-based Convolutional Network (Faster R-CNN)

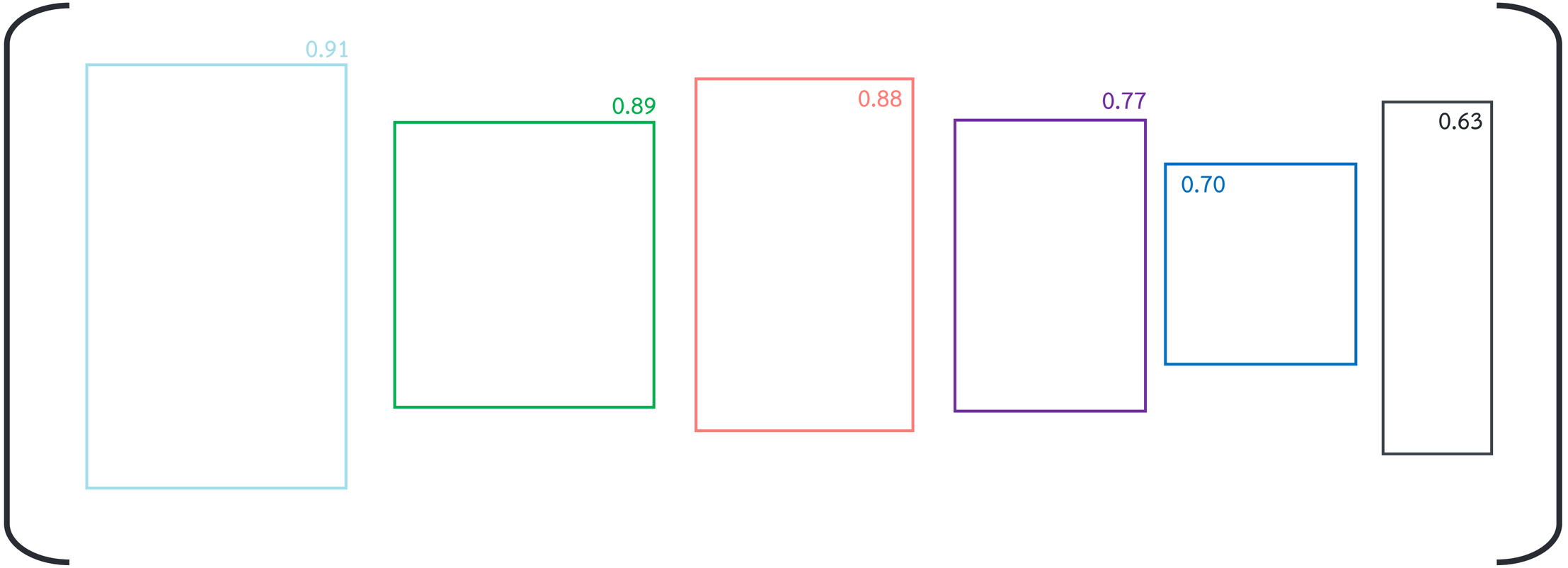
หลักการทำงานของ NMS



# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

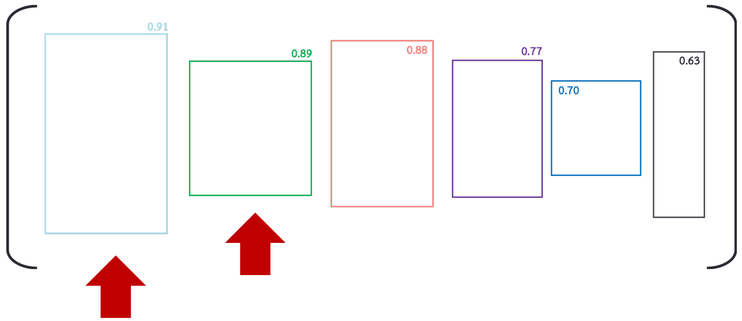
**ขั้นที่ 2.6.1** จัดเรียง Region Proposal ตามค่า Objectness Score จากมากไปน้อย และเก็บไว้ใน List



# Faster Region-based Convolutional Network (Faster R-CNN)

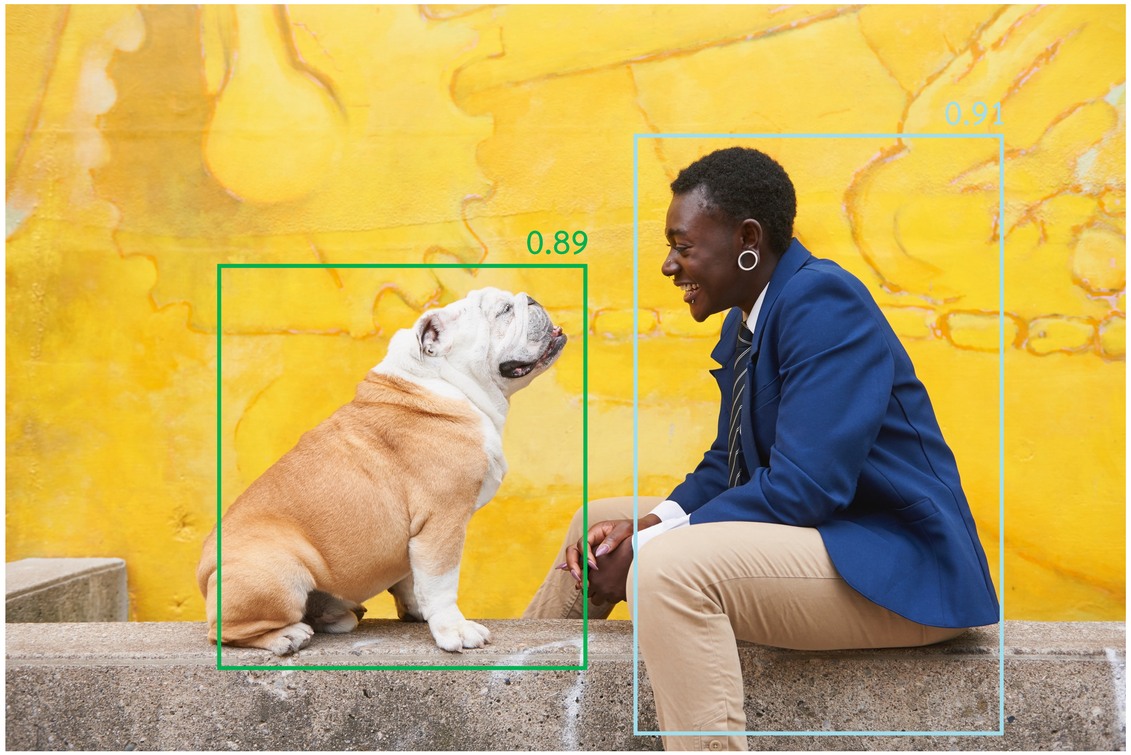
**หลักการทำงานของ NMS**

**ขั้นที่ 2.6.2** หยิบ Region Proposal ออกมาทีละตัวและพิจารณาจากการซ้อนทับกันตามค่า Intersection of Union (IoU) ว่าเกินกว่าค่าที่กำหนดไว้หรือไม่ อาทิเช่น




$$\text{IoU} = \frac{\boldsymbol{R_1 \cap R_2}}{\boldsymbol{R_1 \cup R_2}}$$

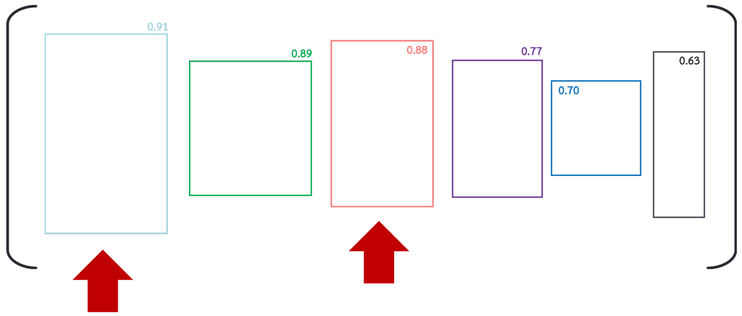
จากกรณีข้างต้นจะพบว่า IoU มีค่าเท่ากับ 0



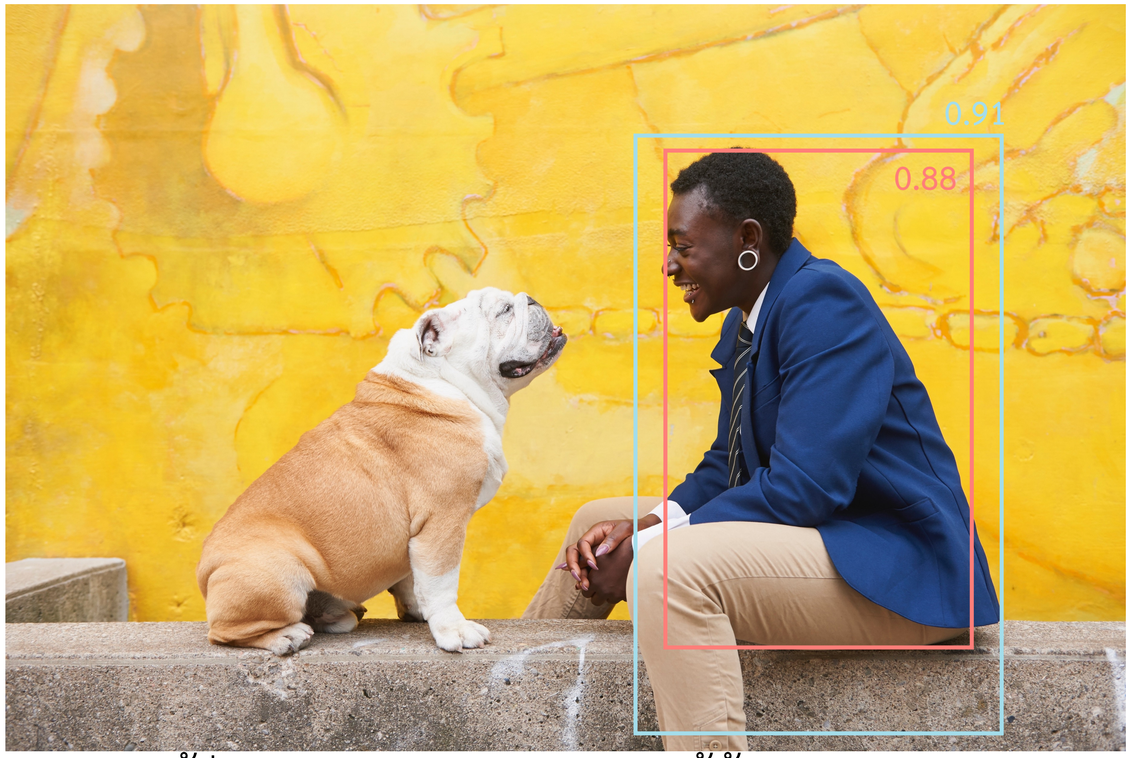
# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

**ขั้นที่ 2.6.2** หยิบ Region Proposal ออกมาทีละตัวและพิจารณาจากการซ้อนทับกันตามค่า Intersection of Union (IoU) ว่าเกินกว่าค่าที่กำหนดไว้หรือไม่ อาทิเช่น




$$\text{IoU} = \frac{\boldsymbol{R_1 \cap R_2}}{\boldsymbol{R_1 \cup R_2}}$$

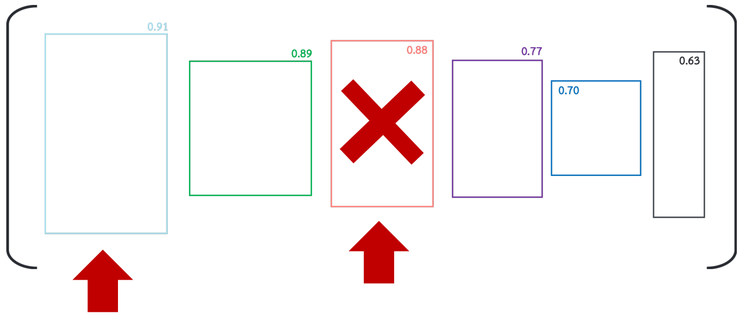



จากกรณีข้างต้นจะพบว่า IoU มีค่าค่อนข้างมาก ดังนั้น จึงสรุปได้ว่า 2 Region Proposal ได้ซ้อนทับวัตถุบริเวณเดียวกัน

# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

**ขั้นที่ 2.6.3** เก็บ Region Proposal ที่มีค่ามากเอาไว้และลบ Region Proposal ที่มีค่าน้อยออก



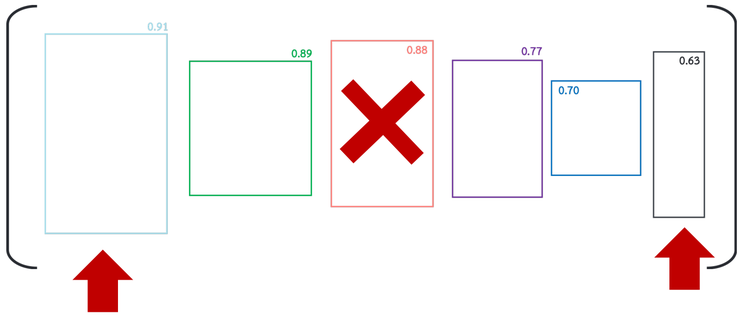

$$\text{IoU} = \frac{\boldsymbol{R_1 \cap R_2}}{\boldsymbol{R_1 \cup R_2}}$$



# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

**ขั้นที่ 2.6.3** เก็บ Region Proposal ที่มีค่ามากเอาไว้และลบ Region Proposal ที่มีค่าน้อยออก



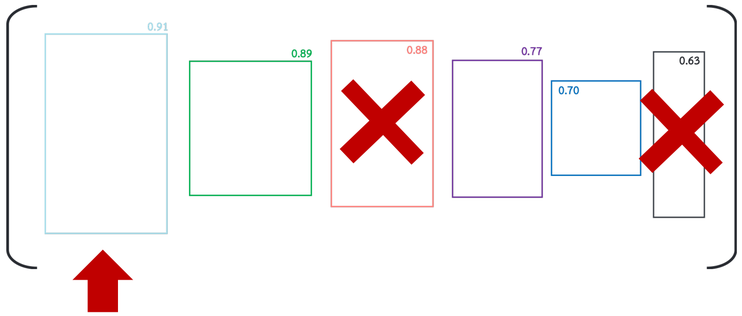

$$\text{IoU} = \frac{\boldsymbol{R_1 \cap R_2}}{\boldsymbol{R_1 \cup R_2}}$$



# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

**ขั้นที่ 2.6.3** เก็บ Region Proposal ที่มีค่ามากเอาไว้และลบ Region Proposal ที่มีค่าน้อยออก



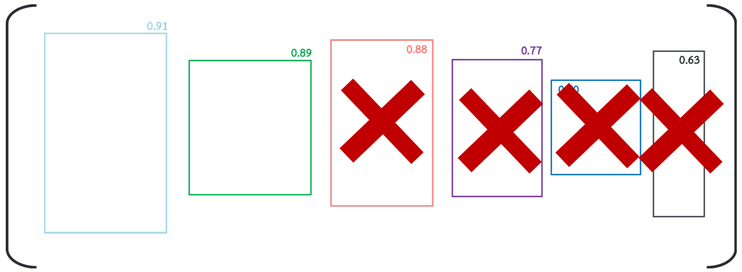

$$\text{IoU} = \frac{\boldsymbol{R_1 \cap R_2}}{\boldsymbol{R_1 \cup R_2}}$$

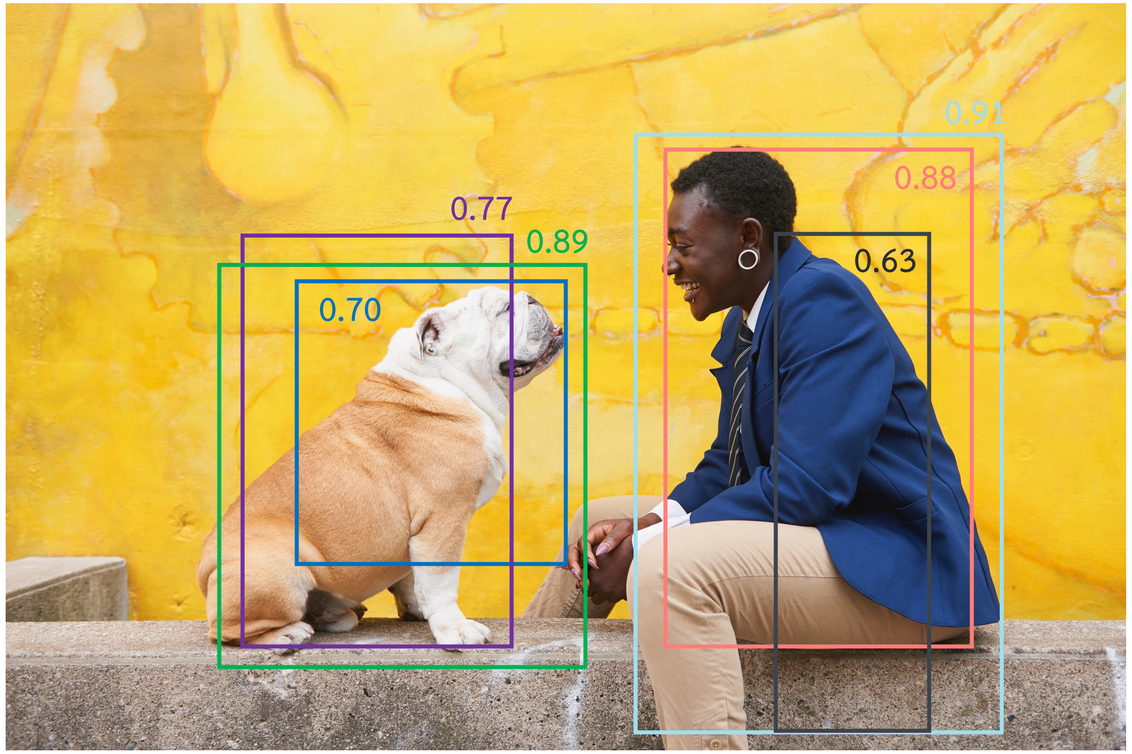


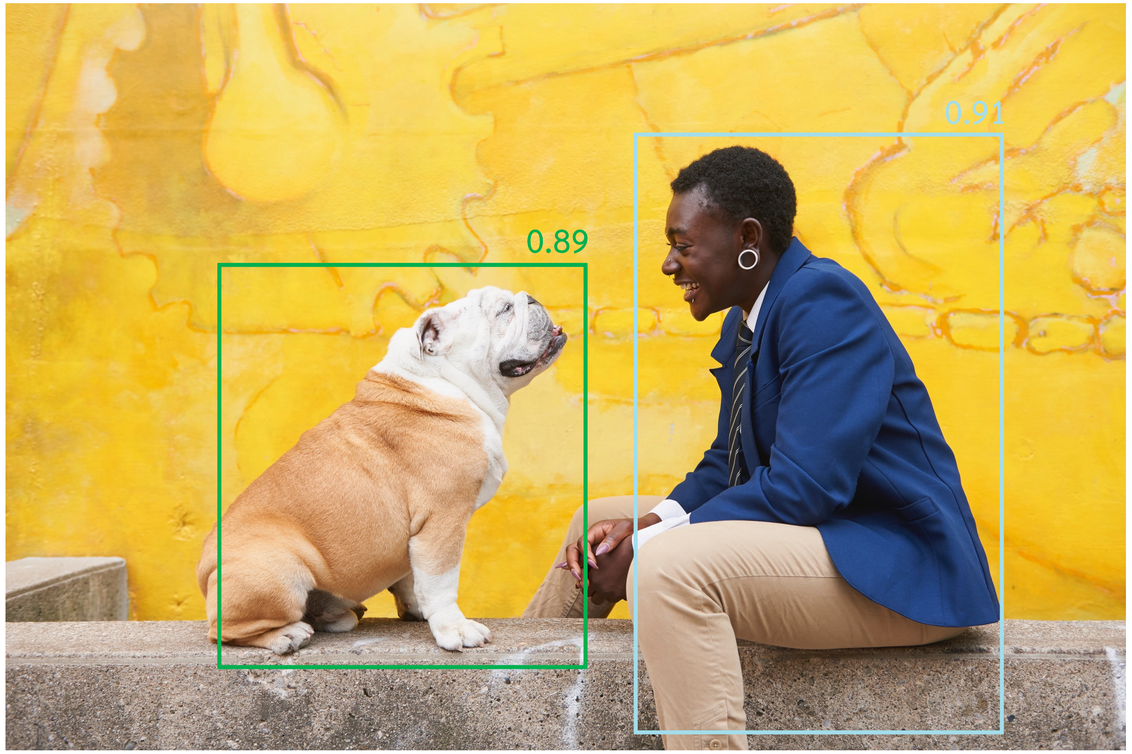
# Faster Region-based Convolutional Network (Faster R-CNN)

**หลักการทำงานของ NMS**

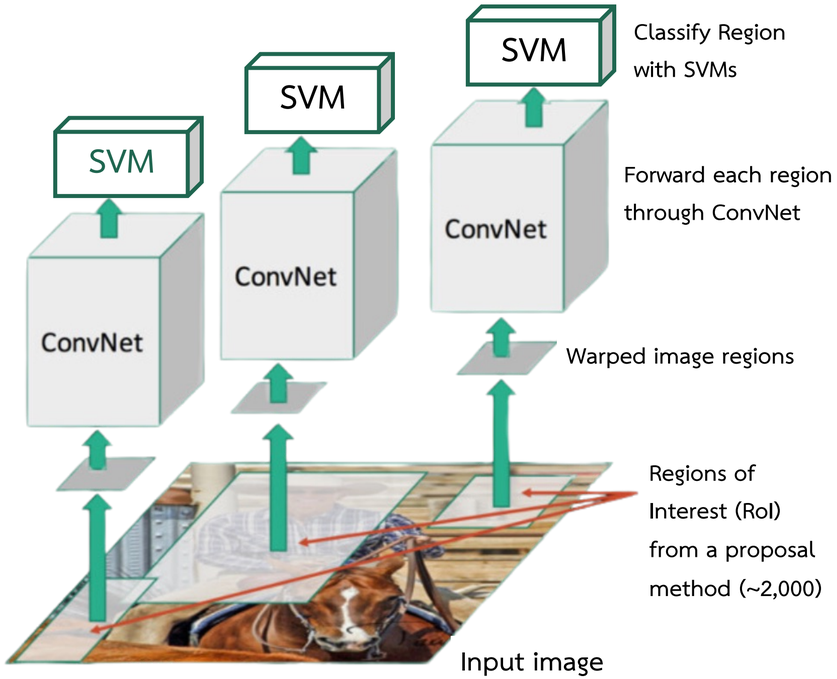
**ขั้นที่ 2.6.4** ทำซ้ำขั้นตอนที่ 2.6.2 ถึง 2.6.3 จนกว่าจะเปรียบเทียบหมด



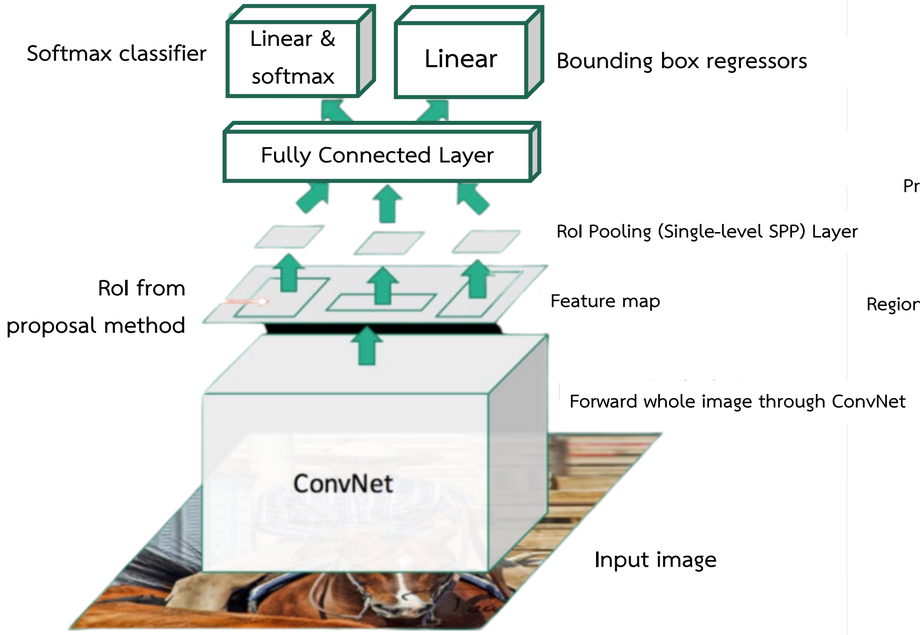



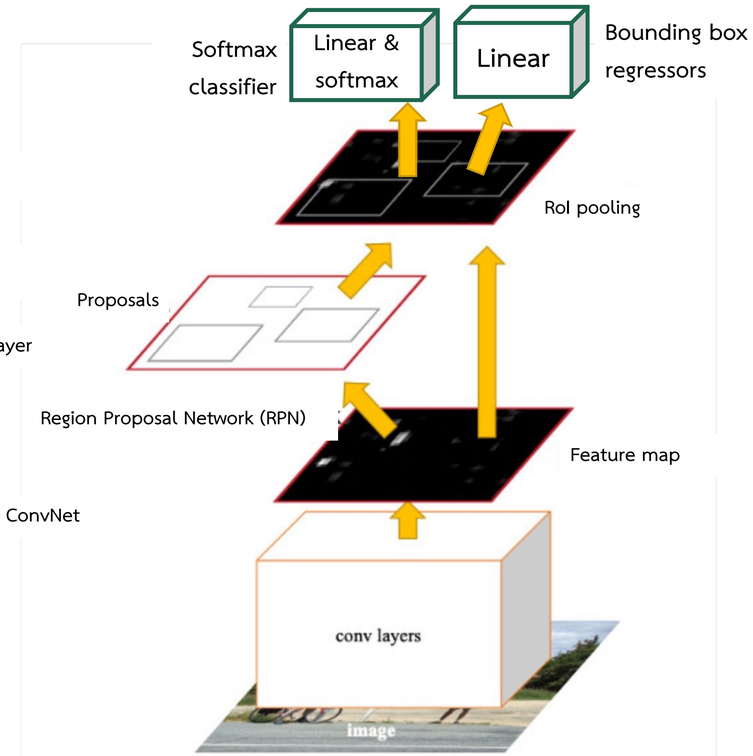
# Faster Region-based Convolutional Network (Faster R-CNN)




(a) R-CNN




(b) Fast R-CNN




(c) Faster R-CNN



# Faster Region-based Convolutional Network (Faster R-CNN)

| | RCNN | Fast RCNN | Faster RCNN |
|---|---|---|---|
| Test time per image with Proposals | 50 seconds | 2 seconds | 0.2 seconds |
| (Speedup) | 1x | 25x | 250x |
| mAP (PASCAL VOC 07) | 66.0 | 66.9 | 66.9 |

อ้างอิง https://towardsdatascience.com/understanding-fast-r-cnn-and-faster-r-cnn-for-object-detection-adbb55653d97



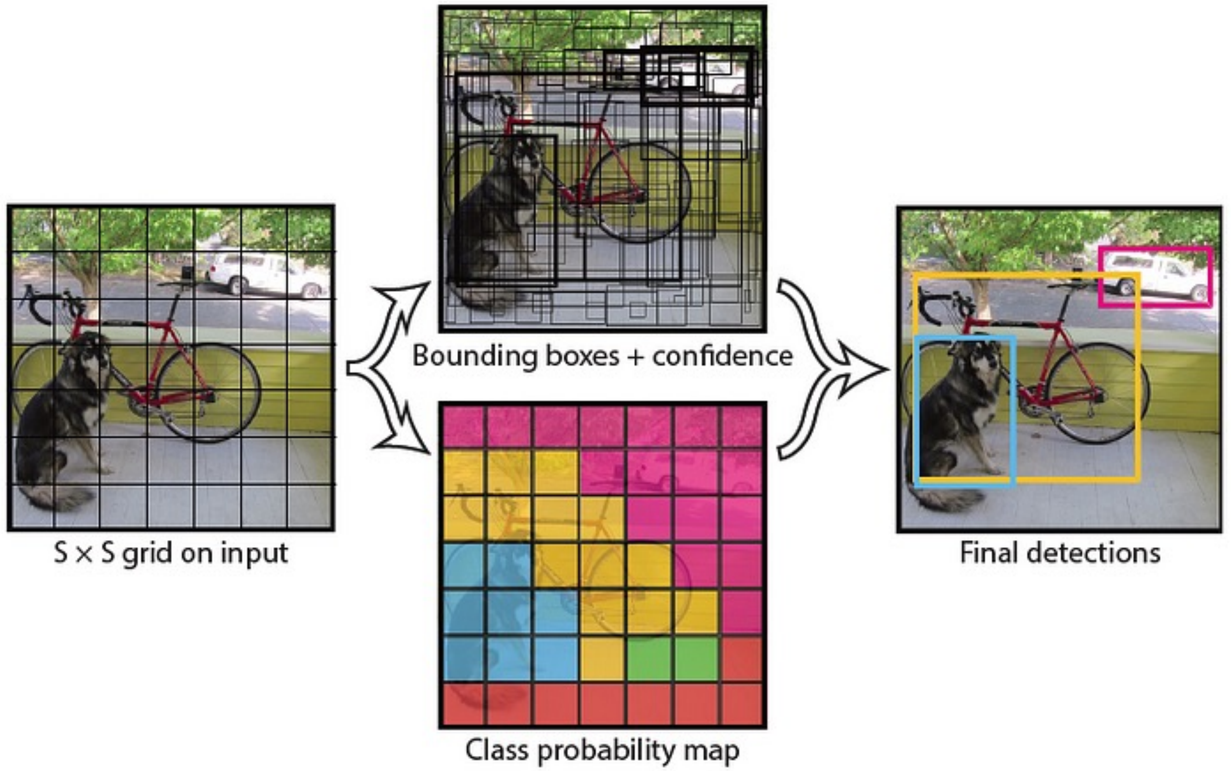
# หัวข้อ

- Multi-Stage Detectors
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- **Single-Stage Detectors**
  - You Only Look Once (YOLO)
- การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics

# You Only Look Once (YOLO)

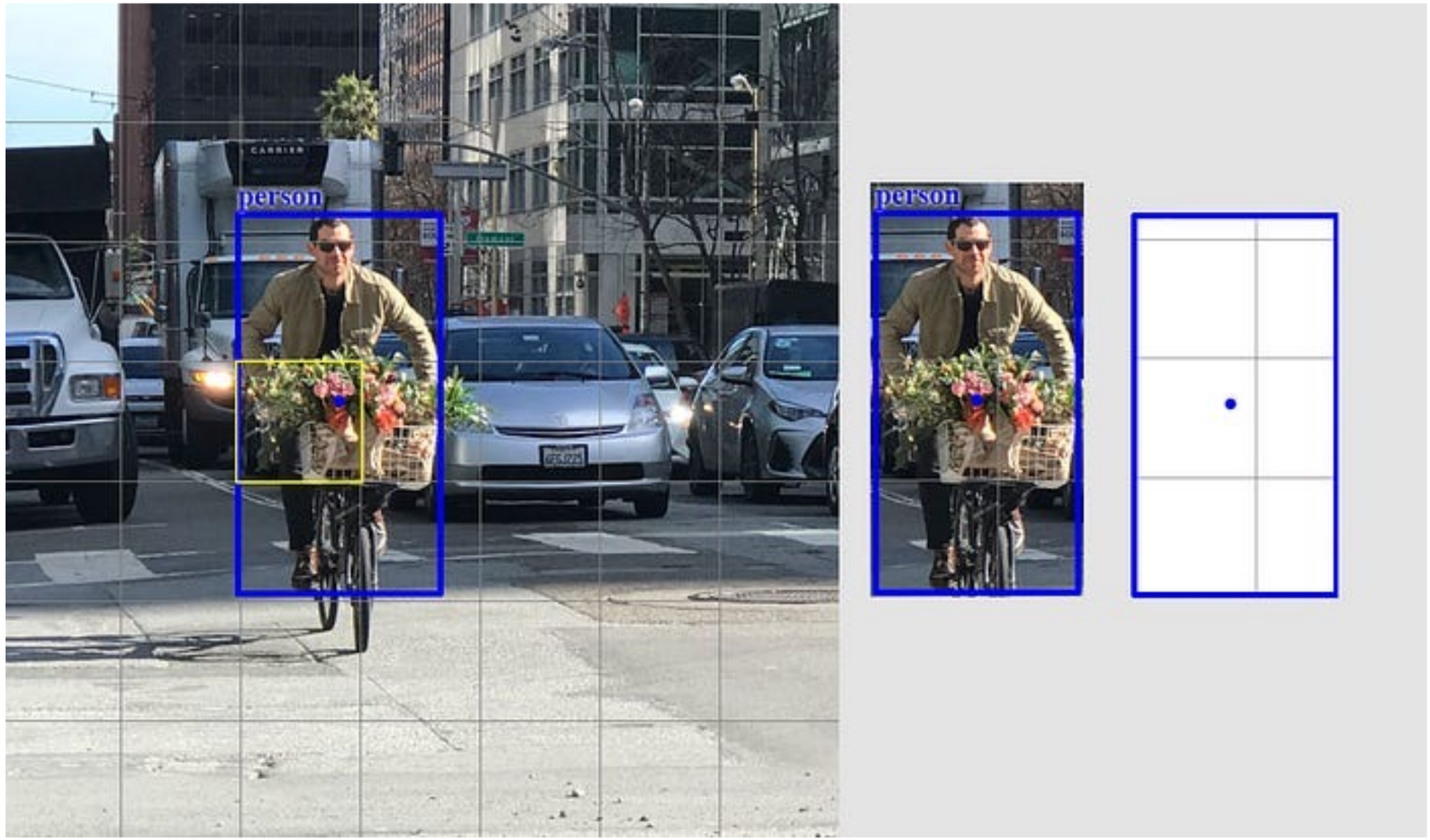
**หลักการทำงานของ YOLO**

สถาปัตยกรรมของ YOLO ได้นำเสนอ Unified Network เพื่อให้เครือข่ายสามารถทำงานในรูปแบบ End-to-end Training Network




อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

# You Only Look Once (YOLO)




**หลักการทำงานของ YOLO**

**ขั้นที่ 1** แบ่งภาพออกเป็น Grid ขนาด S x S โดยแต่ละช่องของ Grid จะทำนายวัตถุจำนวน n อย่าง

ตัวอย่าง ช่องของ Grid สีเหลือง พยายามทำนายวัตถุ “คน” ที่จุดกึ่งกลางอยู่ในช่องของ Grid ดังกล่าว

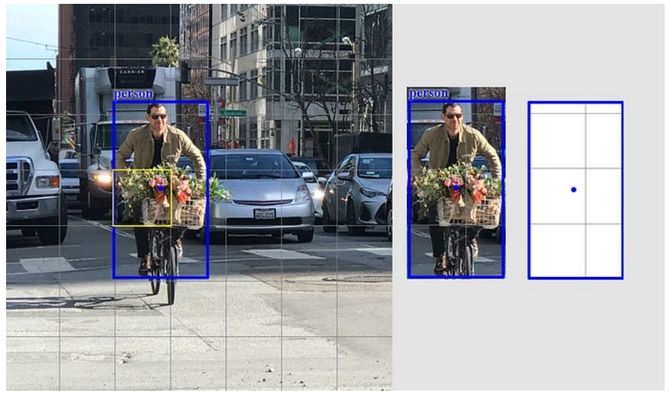
อ้างอิง https://jonathan-hui.medium.com/real-time-object-detection-with-yolo-yolov2-28b1b93e2088

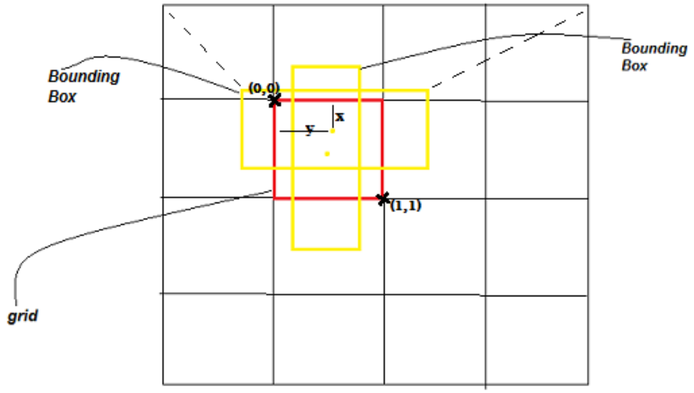
## ความสัมพันธ์ระหว่างวัตถุ--เซลล์–กล่อง-คลาส

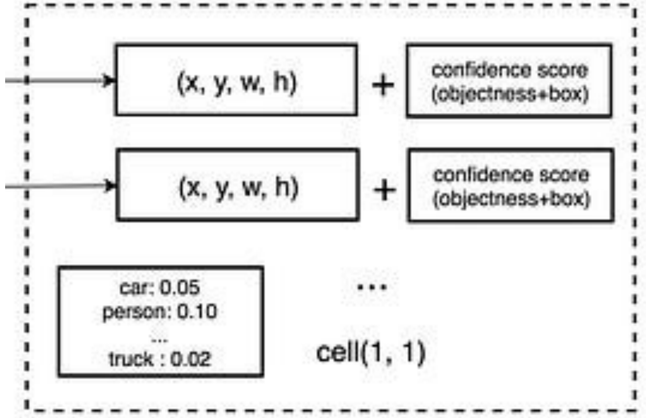
- วัตถุ-กับ-เซลล์ : วัตถุหนึ่งชิ้นถูกกำหนดให้กับเซลล์ที่มีจุดกึ่งกลางของวัตถุ ตกอยู่
- วัตถุ-กับ-กล่อง : ในเซลล์นั้น เลือก 1 กล่องจาก $B$ กล่องที่มี IoU กับ ground truth สูงสุดให้ “เป็น” วัตถุที่ทำนายนั้น ในขณะที่ อีกกล่องจะถูกกำหนดให้เป็นวัตถุอื่นหรือเป็นพื้นหลัง ซึ่งส่วนใหญ่ ในYolo V1 มักกำหนดให้ $B$ มีค่าเท่ากับ 2
- เซลล์-กับ-คลาส : หนึ่งเซลล์มีได้เพียง “หนึ่งคลาส” หากมีหลายวัตถุคนละชนิดอยู่ในเซลล์เดียวกันจะทำให้ Yolo มีประสิทธิภาพไม่ดี (หรือกล่าวคือ class prob ร่วมกันต่อเซลล์) นอกจากนี้ วัตถุเล็กจำนวนมากที่กระจุกในบางบริเวณ สถาปัตยกรรม Yolo V1 ยังทำงานได้ไม่ดีมาก เนื่องจากเซลล์มีขนาดใหญ่

ถ้าพิจารณาจาก cell (1,1) ในกรอบเส้นประของรูป จะพบว่าโมเดลจะให้ผลลัพธ์ดังต่อไปนี้

- **กล่องที่ 1:** $(x, y, w, h)$ และ confidence
- **กล่องที่ 2:** $(x, y, w, h)$ และ confidence
- **ตารางคะแนนคลาส** (เช่น car 0.05, person 0.10, truck 0.02, ...) ซึ่งเป็น $P(class \mid object)$ เวลานำไปใช้งานจะคูณคลาสกับ confidence ของแต่ละกล่อง ทำให้ได้คะแนนต่อคลาสต่อกล่อง ก่อนนำไปคัดกรองด้วยวิธีการ NMS จากนั้น จึงนำผลลัพธ์ที่ได้เป็น detection สุดท้าย



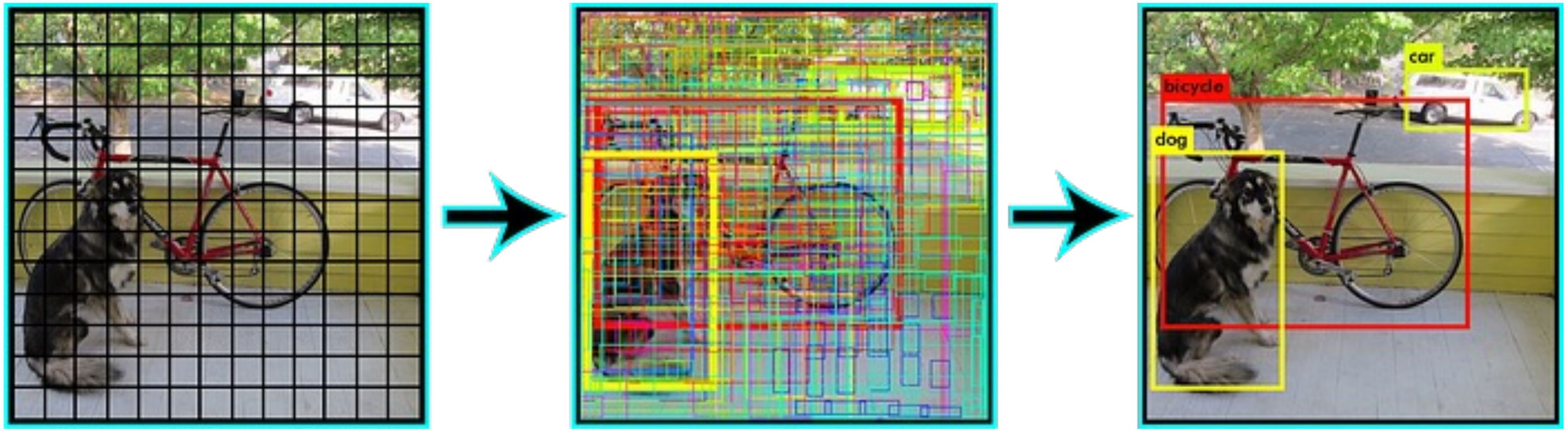






อ้างอิง https://jonathan-hui.medium.com/real-time-object-detection-with-yolo-yolov2-28b1b93e2088

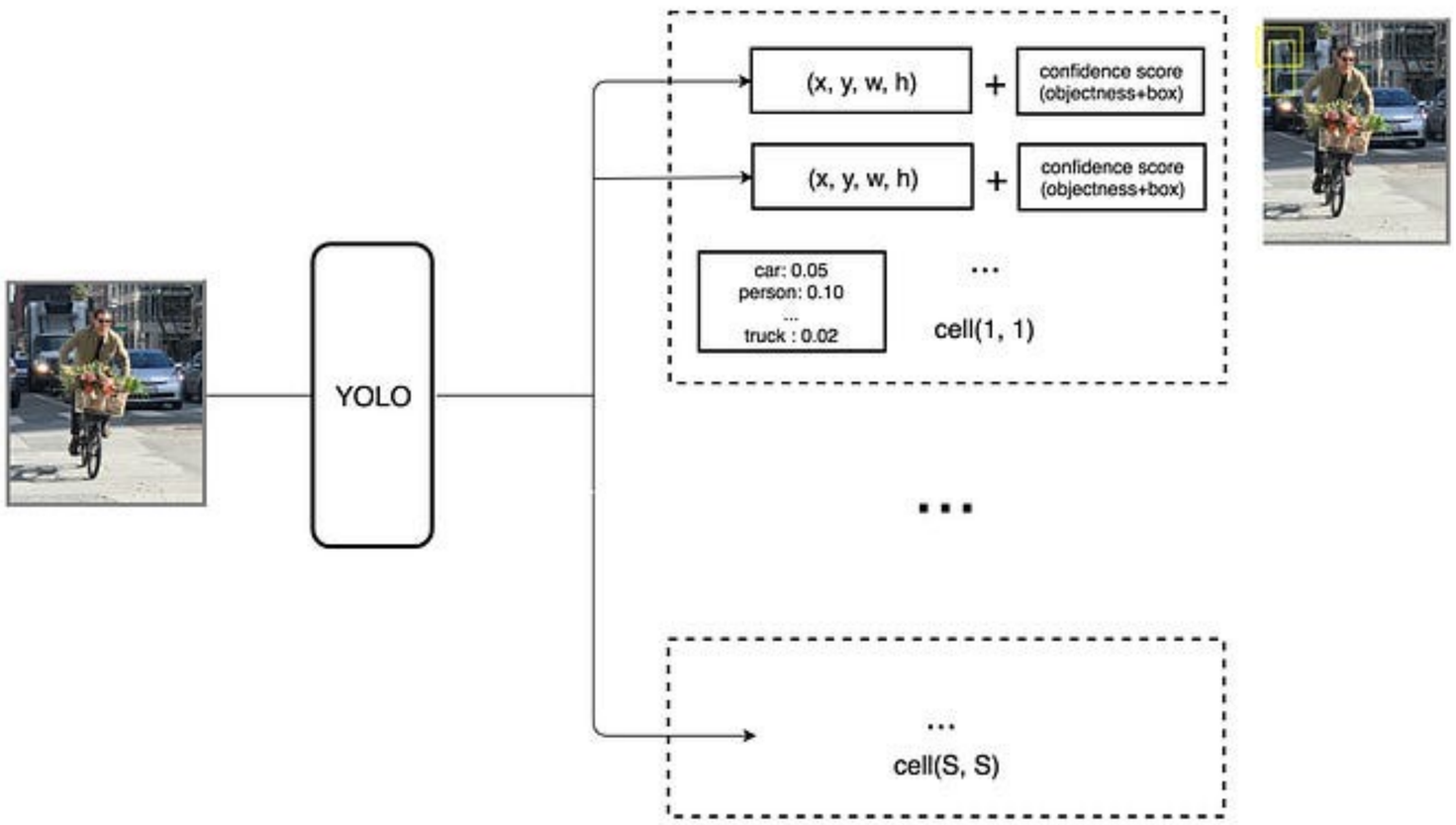
# You Only Look Once (YOLO)




YOLO จัดการกับ Bounding Box ที่ซ้ำซ้อนกันโดยอาศัยเทคนิค non-maximal suppression เพื่อขจัด Bounding Box ที่ซ้ำซ้อนกันออก โดยพิจารณาจากค่า Confidence Score จากผลการวิจัยพบว่าเทคนิค Non-Maximal Suppression สามารถช่วยเพิ่มประสิทธิภาพได้ประมาณ 2- 3% ในตัวชี้วัด mAP

อ้างอิง อ้างอิง https://jonathan-hui.medium.com/real-time-object-detection-with-yolo-yolov2-28b1b93e2088

# You Only Look Once (YOLO)



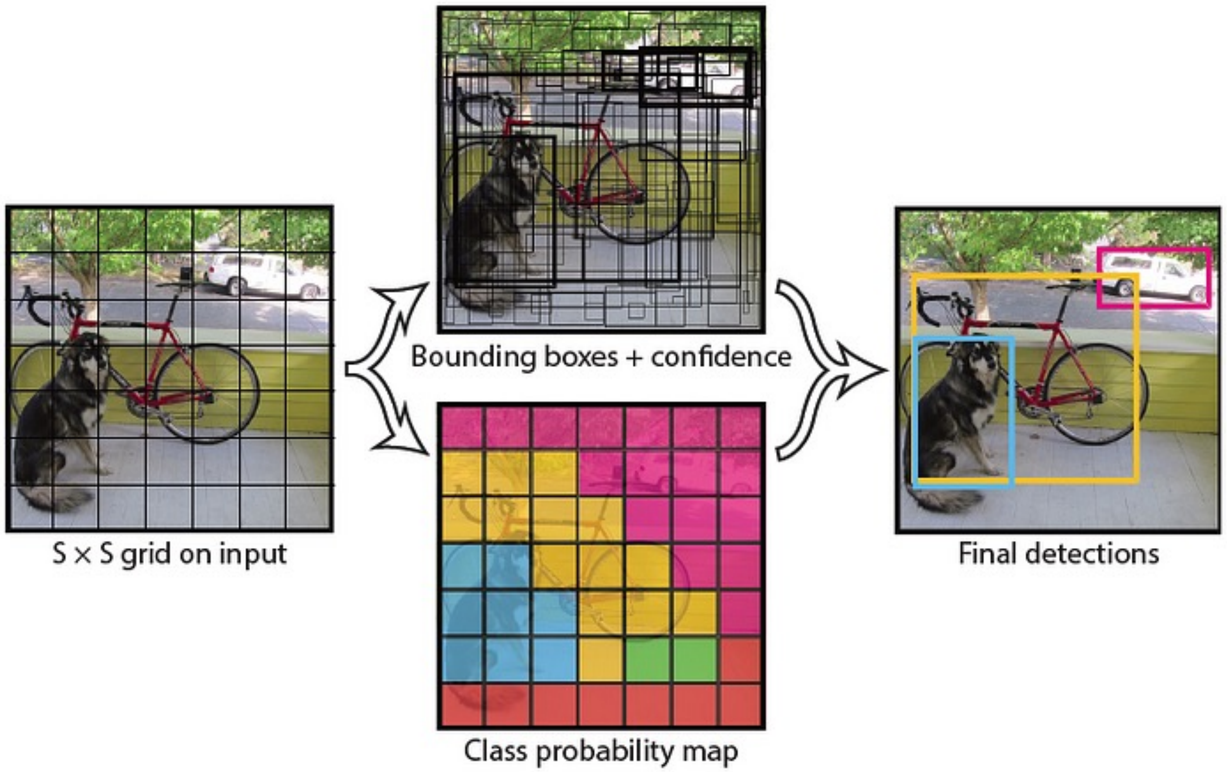

จากภาพจะพบว่า Boundary Box หนึ่งจะประกอบด้วย 2 ส่วน ได้แก่ **(x, y, w, h)** และ **ค่า Box Confidence Score (Objectness)** ซึ่งค่า Objectness สะท้อนถึงความเป็นไปได้ที่ Boundary Box จะมีวัตถุอยู่และความแม่นยำของกล่องนั้น จากนั้น (1) นำค่าของ w และ h ของ Boundary Box ไป Normalize ด้วยความสูง (H) และความกว้างของภาพ (W) ถัดมา (2) ค่า x และ y จะใช้เป็นค่าชดเชยในการปรับตำแหน่งของ Boundary Box ดังนั้น x, y, w และ h จะอยู่ในช่วง 0 ถึง 1 สุดท้าย ในแต่ละ Cell (ไม่ใช้แต่ละ Boundary Box) จะบอกค่าความน่าจะเป็นของแต่ละคลาสดังภาพ

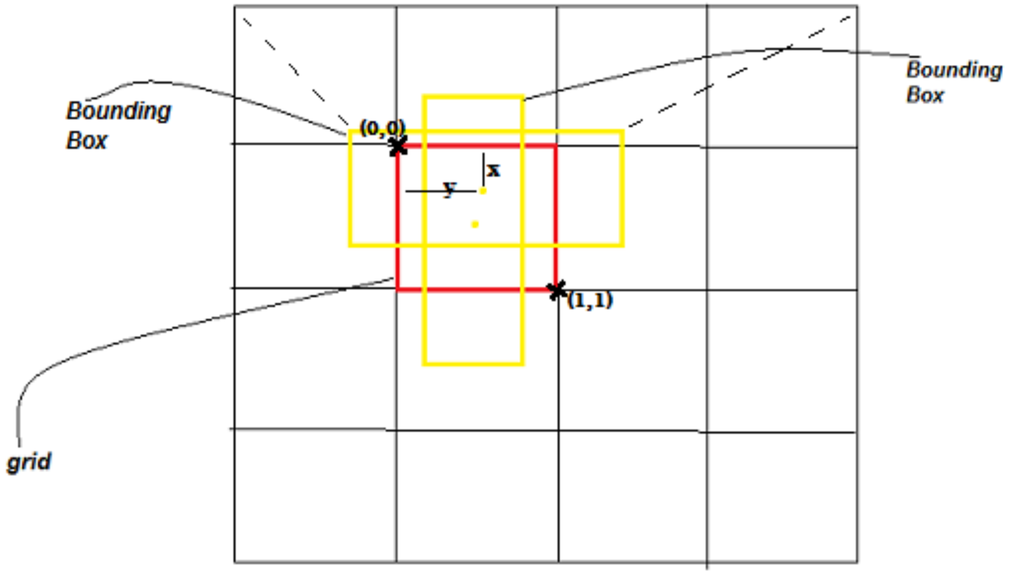
อ้างอิง https://jonathan-hui.medium.com/real-time-object-detection-with-yolo-yolov2-28b1b93e2088

# You Only Look Once (YOLO)

**หลักการทำงานของ YOLO**

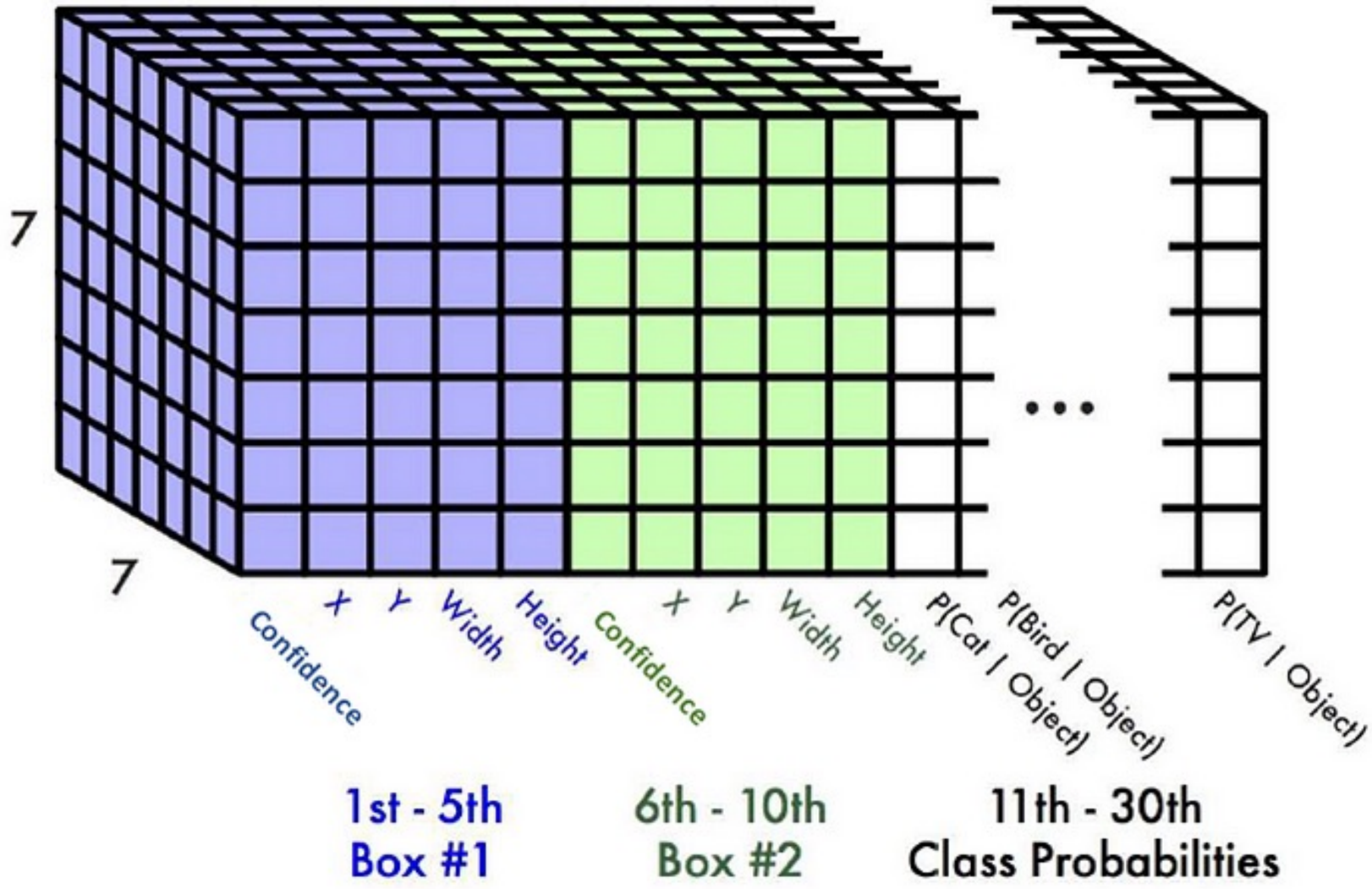
**ขั้นที่ 2** แต่ละ Grid จะสร้าง B (กำหนดให้ B = 2) Bounding Box (ซึ่งประกอบด้วย cx, cy, w, h) และค่าความเชื่อมั่น (Confident) ของแต่ละกล่อง




อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

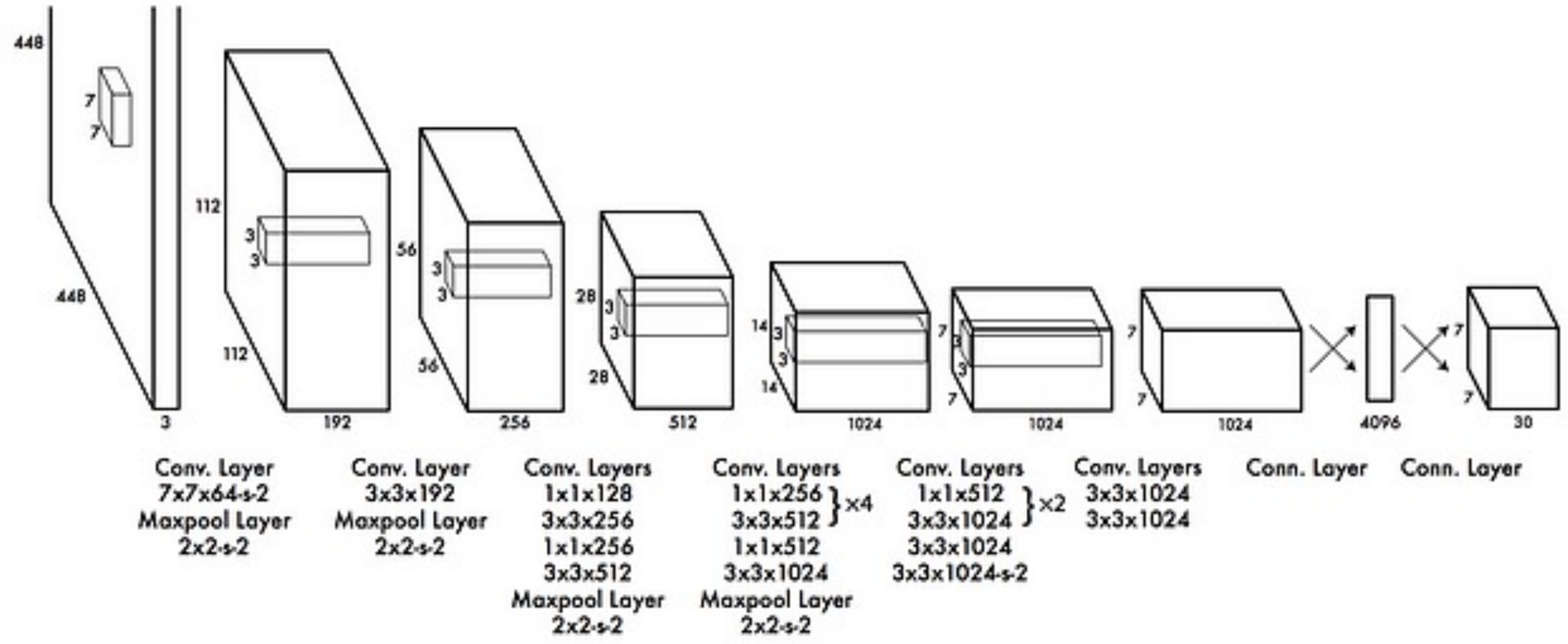
# You Only Look Once (YOLO)




อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

# You Only Look Once (YOLO)

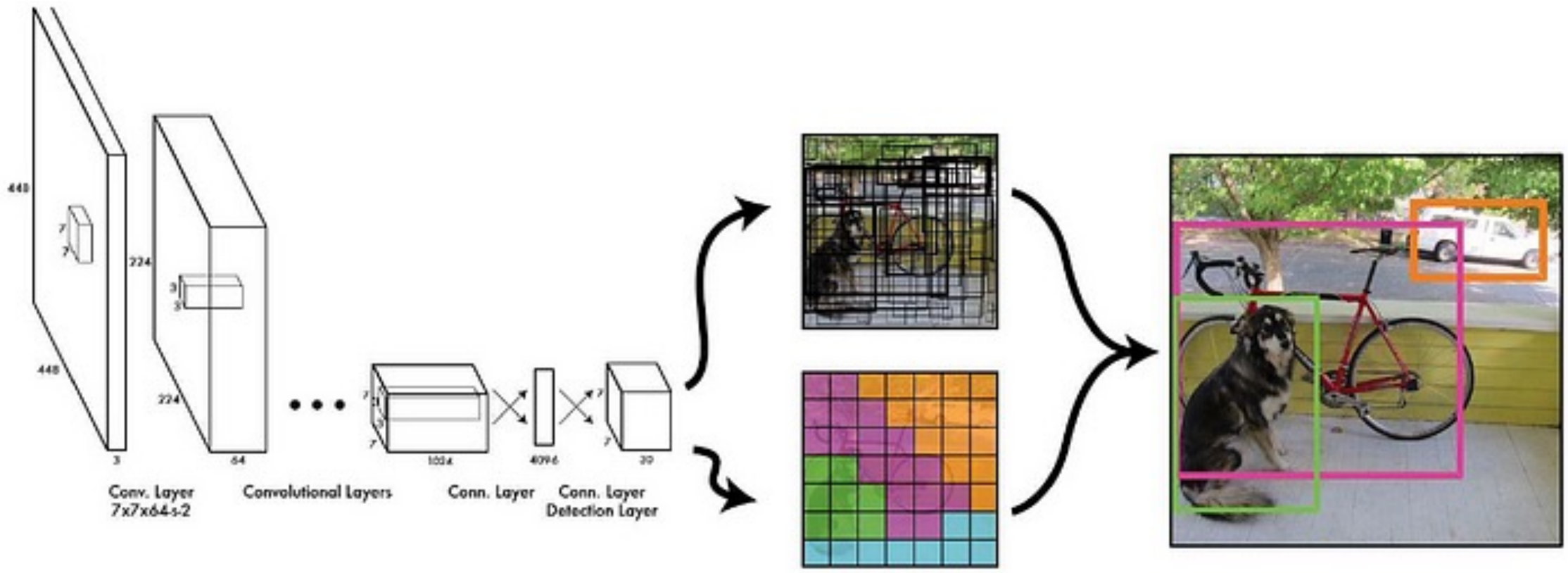
ซึ่งวิธีการ YOLO อาศัยสถาปัตยกรรมเบื้องต้นดังภาพ ซึ่งประกอบไปด้วย 24 convolutional layers และ 2 fully connected layers




อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

# You Only Look Once (YOLO)

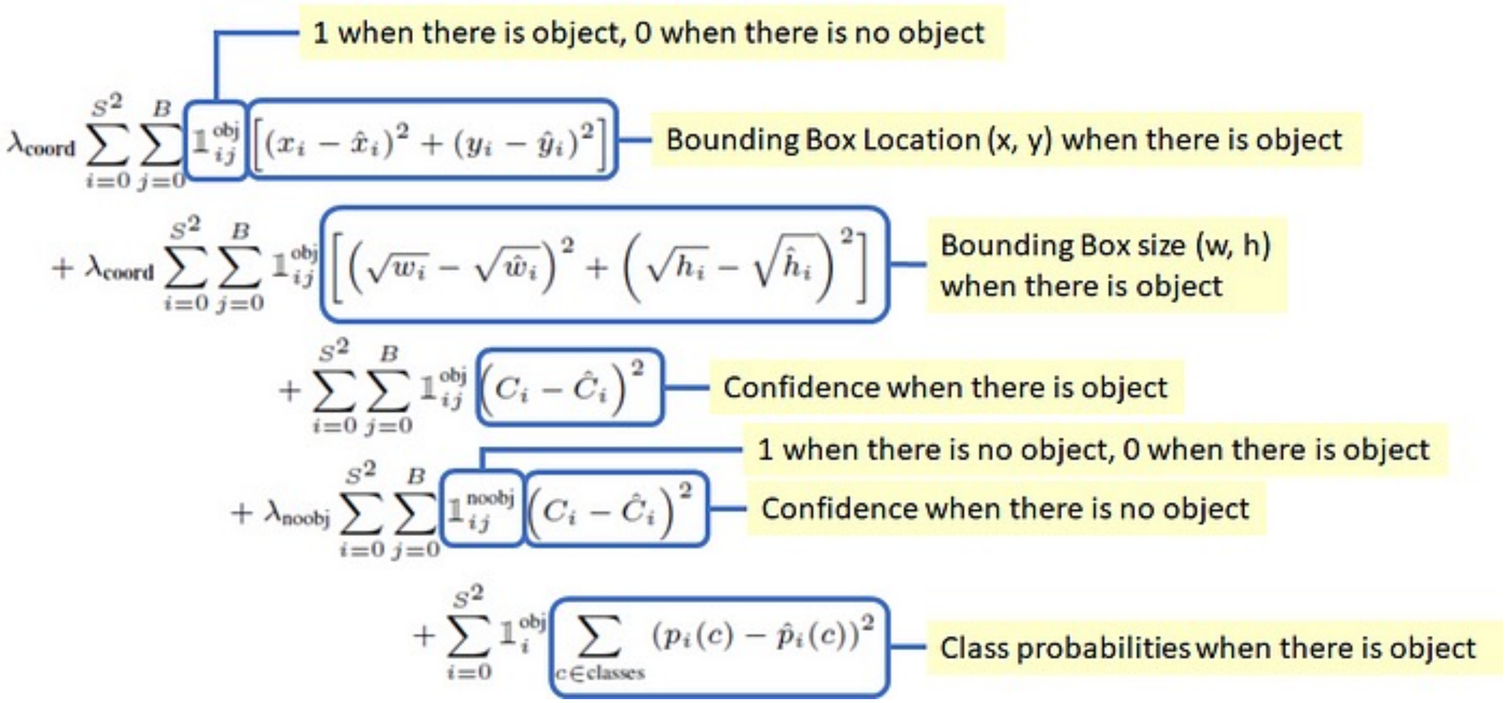
ซึ่งวิธีการ YOLO อาศัยสถาปัตยกรรมเบื้องต้นดังภาพ ซึ่งประกอบไปด้วย 24 convolutional layers และ 2 fully connected layers




อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

# You Only Look Once (YOLO)

## Loss Function




**ประกอบไปด้วย 5 ส่วน ได้แก่**

**ส่วนที่ 1 (x, y)** ใช้พิจารณาตำแหน่งความคลาดเคลื่อนของ bounding box โดยอาศัย sum of square error (SSE)

**ส่วนที่ 2 (w, h)** ใช้พิจารณาความกว้างและความสูงที่อยู่ในช่วง 0 ถึง 1 (The bounding box width and height are normalized by the image width and height) โดยอาศัยหลักการ SSE นอกจากนี้ การที่อาศัยรากที่สองเนื่องจากลดผลกระทบของขนาดกล่องใหญ่กับเล็ก

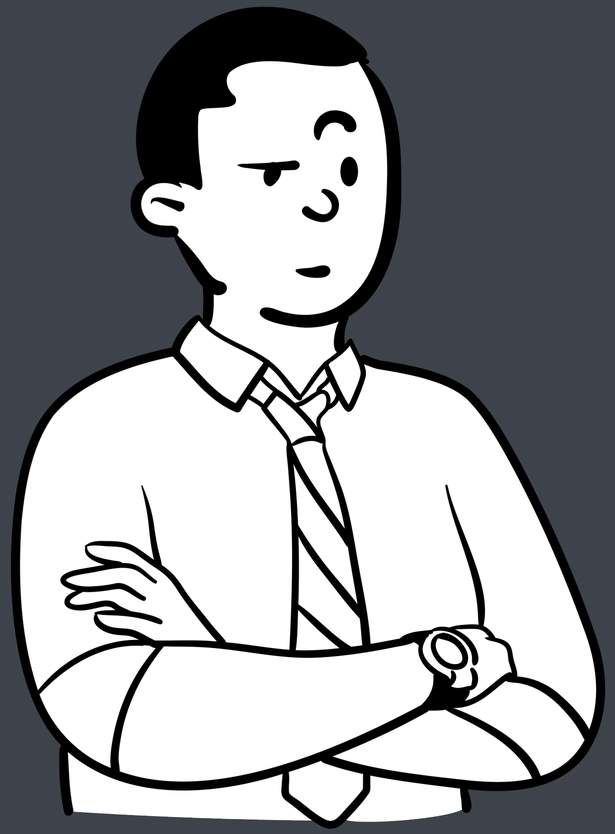
**ส่วนที่ 3 และ 4 (confidence) พิจารณาจากค่า IoU ระหว่าง predicted box และ ground truth** เพื่อลดปัจจัยการทำให้แบบจำลองไม่เสถียนและลดค่าความมั่นใจกรณีไม่มี Object นอกจากนี้ ค่า **λnoobj** จะกำหนดให้เป็น 0.5

**ส่วนที่ 5 (Class Probabilities)**: ค่า SSE ของค่าความน่าจะเป็นแต่ละคลาส

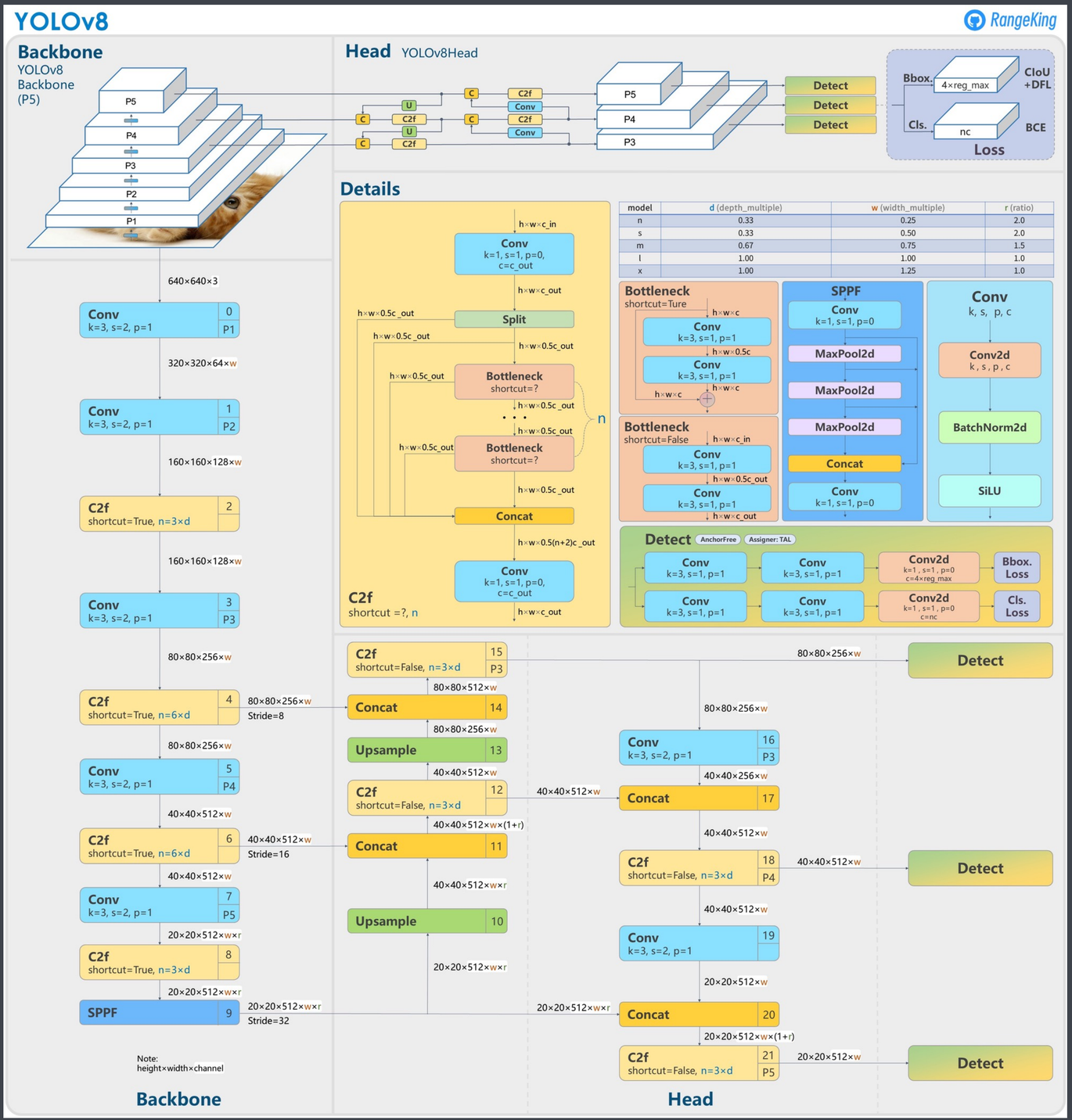
หมายเหตุ ค่า **λ**coord กำหนดเท่ากับ 5 เพื่อเพิ่มค่า loss สำหรับตำแหน่งของ bounding box ที่ทำนายได้

อ้างอิง https://towardsdatascience.com/yolov1-you-only-look-once-object-detection-e1f3ffec8a89

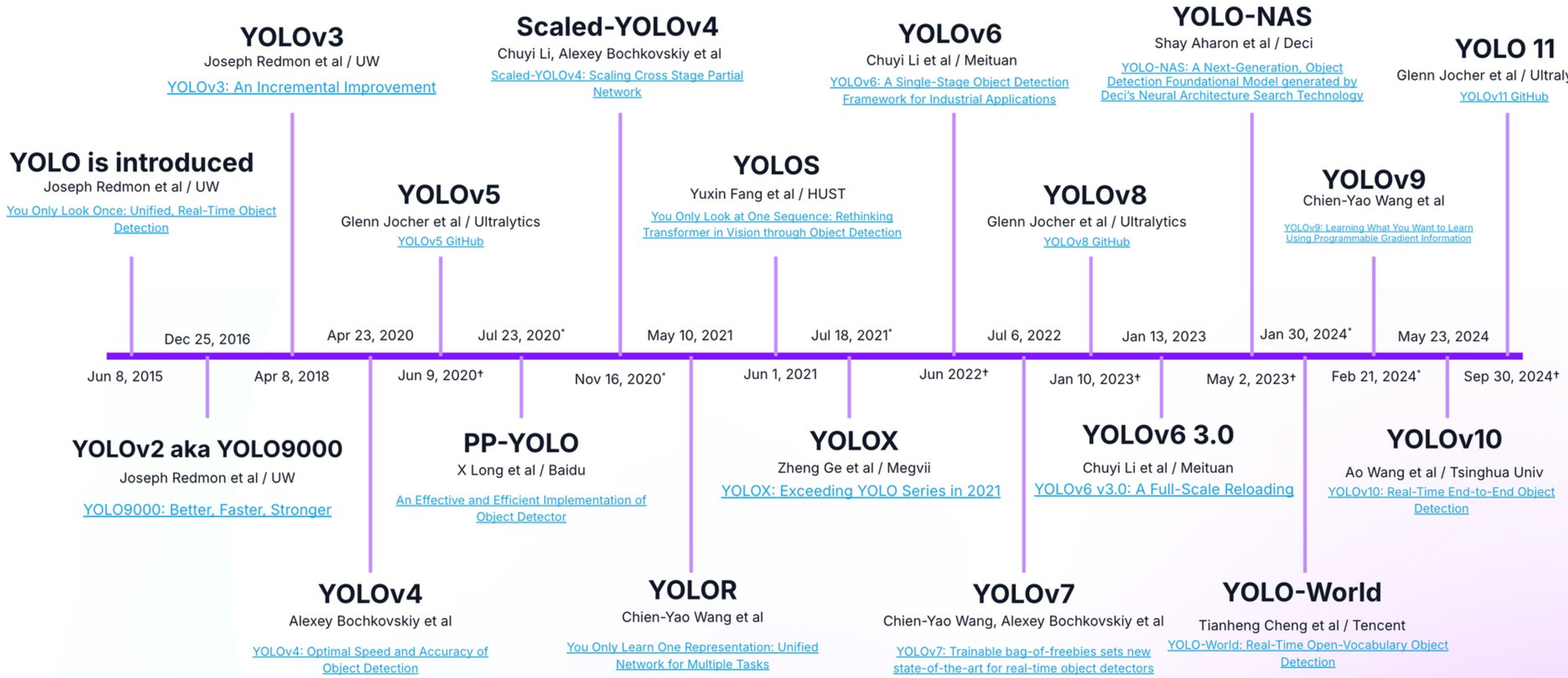
# You Only Look Once
# Ver. 8 (YOLOv8)



อ้างอิง https://blog.roboflow.com/whats-new-in-yolov8/



# YOLO Greatest Hits




*Denotes paper updated after first publication date

†Denotes repository predates paper publication date

อ้างอิง https://blog.roboflow.com/guide-to-yolo-models/



# หัวข้อ

- Multi-Stage Detectors
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- Single-Stage Detectors
  - You Only Look Once (YOLO)
- **การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics**

# ชุดคำสั่ง Ultralytics YOLOv8 คืออะไร




Ultralytics YOLOv8 เป็น Open Source ของบริษัท Ultralytics Inc. ที่ให้บริการผ่านทาง GitHub ที่พัฒนามาจาก YOLO เวอร์ชันก่อนหน้า ซึ่งสามารถนำไปใช้กับงาน  Object Detection, Instance Segmentation, Image Classification และงาน Pose Estimation ได้



# การใช้งานชุดคำสั่ง Ultralytics YOLOv8

- **วิธีการติดตั้ง**

  ความต้องการของ Environment ต้องใช้ Python>=3.8 และ PyTorch>=1.8

```
pip install ultralytics
```

- **วิธีใช้งานเบื้องต้น**

  Command Line Interface (CLI)

```
yolo predict model=yolov8n.pt source='https://ultralytics.com/images/bus.jpg'
```

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง ประกอบไปด้วย 6 ขั้นตอน ดังนี้

| ข้อ | คำอธิบาย |
|---|---|
| 1 | สร้างไฟล์ dataset.yaml สำหรับกำหนดรายละเอียดของชุดข้อมูล |
| 2 | สร้าง Label ตามฟอร์แมต YOLO |
| 3 | จัดเรียง Directories ไฟล์ให้ถูกต้อง |
| 4 | เลือกโมเดที่จะนำไปใช้ |
| 6 | การฝึกสอนโมเดล |
| 5 | การทดสอบและวัดผลโมเดล |

อ้างอิง : https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/#option-2-create-a-manual-dataset

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

```
# strongly recommend absolute path

# train images
train: datasets/train/images

# validation images
val: datasets/valid/images

# test images (optional)
test: datasets/test/images

# Replace with the actual number of classes in your dataset
nc: 1

# Replace with the actual class names
names: ['car']
```

**1. สร้างไฟล์ dataset.yaml สำหรับกำหนดรายละเอียดของชุดข้อมูล**

ในชุดคำสั่ง Ultralytics YOLOv8 ใช้การกำหนดลักษณะข้อมูลโดยใช้ไฟล์ yaml สำหรับการเขียนไฟล์ .yaml จำเป็นต้องตัวแปรดังต่อไปนี้

- **train** เป็นที่ของภาพสำหรับการฝึกฝน
- **val** เป็นที่อยู่ของภาพสำหรับการ Validation
- **test** เป็นที่อยู่ของภาพสำหรับการทดสอบ
- **nc** เป็นการกำหนดจำนวนคลาสทั้งหมดในชุดข้อมูล
- **names** เป็นชื่อของคลาสทั้งหมดที่อยู่ในรูป List

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง




zidane.txt

```
0  0.481719 0.634028 0.690625 0.713278
0  0.741094 0.524306 0.314750 0.933389
27 0.364844 0.795833 0.078125 0.400000
```

class, x_center, y_center, width, height

หมายเหตุ คลาสที่ 0 จะนำเสนอด้วยกรอบสีส้ม ขณะที่ คลาสที่ 27 จะนำเสนอด้วยสีเขียว โดยตำแหน่งของ YOLO จะประกอบด้วย x_center, y_center, width, height

**2. ทำแบบ Label ให้อยู่ในรูปแบบของ YOLO**

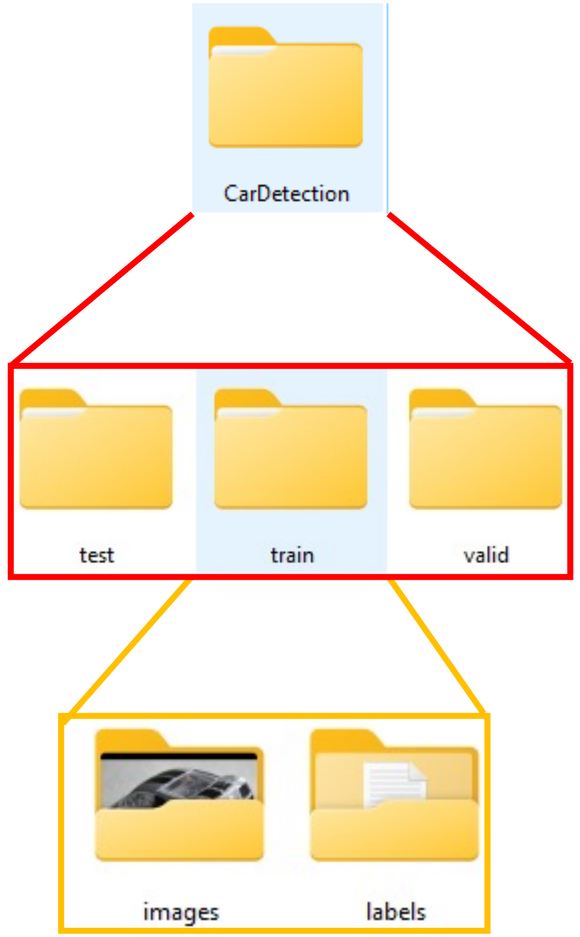
ฉลากข้อมูลสำหรับ YOLO จะอยู่ในรูปแบบของไฟล์ .txt (1 ไฟล์เฉลยสำหรับต่อ 1 ภาพ) โดยที่ ข้อมูลภายในไฟล์จะมีลักษณะการจัดเก็บเป็น “หนึ่งแถวต่อวัตถุหนึ่งชิ้น” และใน 1 แถว จะประกอบด้วยข้อมูลดังนี้

- ประเภท (Class),
- จุด x_center,
- จุด y_center,
- ความกว้างของวัตถุ
- ความสูงของวัตถุ

จากนั้น ตำแหน่งจะต้องถูก Normalize ให้อยู่รูปแบบ 0 – 1 โดยใช้ขนาดของภาพ ซึ่งมุมบนซ้ายของภาพ แทน จุด (0, 0) และมุมล่างขวา แทน จุด (1, 1)

ที่มาของภาพ : https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/#22-create-labels

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง




**3. จัดเรียง Directories ไฟล์ให้ถูกต้อง**

เมื่อกำหนดให้ตัวแปร path คือ ./datasets/coco128 และ train คือ images/train2017 ระบบจะระบุตำแหน่งของ label ให้เองอัตโนมัติโดยระบบจะเปลี่ยนจาก /images/ ไปเป็น /labels/ ให้อัตโนมัติ ตัวอย่างเช่น
ตำแหน่งของรูป คือ

```
./datasets/train/images/im0.jpg
```

ระบบจะหาตำแหน่งเฉลยโดยใช้ที่อยู่ให้อัตโนมัติ

```
./datasets/train/labels/im0.txt
```

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

Performance

Detection (COCO) | Detection (Open Images V7) | Segmentation (COCO) | Classification (ImageNet) | Pose (COCO) >

See Detection Docs for usage examples with these models trained on COCO, which include 80 pre-trained classes.

| Model | size (pixels) | mAP$^{val}$ 50-95 | Speed CPU ONNX (ms) | Speed A100 TensorRT (ms) | params (M) | FLOPs (B) |
|---|---|---|---|---|---|---|
| YOLOv8n | 640 | 37.3 | 80.4 | 0.99 | 3.2 | 8.7 |
| YOLOv8s | 640 | 44.9 | 128.4 | 1.20 | 11.2 | 28.6 |
| YOLOv8m | 640 | 50.2 | 234.7 | 1.83 | 25.9 | 78.9 |
| YOLOv8l | 640 | 52.9 | 375.2 | 2.39 | 43.7 | 165.2 |
| YOLOv8x | 640 | 53.9 | 479.1 | 3.53 | 68.2 | 257.8 |

**4. เลือกโมเดลที่ใช้งาน**

ใน Ultralytics YOLOv8 มีโมเดล YOLO 8 ที่ถูกฝึกฝนด้วย ชุดข้อมูล COCO ให้เลือกใช้จำนวน 5 ขนาด ดังนั้นนักศึกษาสามารถเลือกได้ตามความเหมาะสม เช่น ความเร็ว ความถูกต้อง



# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

**5. การฝึกสอนโมเดล**

วิธีฝึกสอนโมเดล YOLO สามารถทำได้โดยการสร้างไฟล์ .py จากนั้น ให้กำหนดรายละเอียดพารามิเตอร์ต่าง ๆ และฝึกสอนได้เลย

```
from ultralytics import YOLO
import yaml

# Initialize the model
model = YOLO("yolov8n.pt")

# Train the model with your custom dataset
results = model.train(data="mock_data_file.yaml", epochs=5, batch=16, imgsz=256)

print("Training completed")
```

- batch คือ การส่งรูปให้โมเดลครั้งละหลายรูป ซึ่งสามารถส่งไปฝึกสอนหรือทดสอบครั้งละหลายรูปได้

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

**6. การทดสอบและวัดผลโมเดล**

สำหรับวิธีการทดสอบและวัดผล YOLO สามารถทำได้โดยการสร้างไฟล์ .py จากนั้น ใส่ที่อยู่ของโมเดลที่ฝึกสอนแล้ว และเรียกใช้ฟังก์ชัน val() จากโมเดลได้เลย พร้อมกับกำหนดรายละเอียดพารามิเตอร์ต่าง ๆ

```
from ultralytics import YOLO

# Load a model
model = YOLO("runs/detect/train/weights/best.pt")

# Customize validation settings
validation_results = model.val(data="mock_data_file.yaml", imgsz=256, batch=16, conf=0.25,
iou=0.5)
```

- batch คือ การส่งรูปให้โมเดลครั้งละหลายรูป ซึ่งสามารถส่งไปฝึกสอนหรือทดสอบครั้งละหลายรูปได้
- conf คือ confidence score ที่ใช้ขจัด Object ที่มีความมั่นใจต่ำออกไป โดยปกติโปรแกรมได้กำหนดค่าเริ่มต้นเป็น 0.001 (ไม่แนะนำ)
- iou คือ เกณฑ์ในการพิจารณาการซ้อนทับกันของวัตถุในขั้นตอน non-maximize suppression โดยปกติโปรแกรมได้กำหนดค่าเริ่มต้นเป็น 0.6

# วิธีการฝึกฝน Ultralytics YOLOv8 ด้วยชุดข้อมูลกำหนดเอง

**6. การทดสอบและวัดผลโมเดล**

```
img = cv2.imread(img_path)

# Run inference
results = model(img)

# Process results list
for result in results:
    # Save the result image with predictions
    output_img_path = os.path.join(output_path, img_name)
    result.save(output_img_path)
```

เนื่องจากชุดคำสั่ง Ultralytics YOLOv8 รองรับการทำงานทั้ง object detection และ segmentation ส่งผลให้เวลาคืนค่าตัวแปร result จะให้ผลลัพธ์ออกมาทั้ง 2 แบบ ดังนั้น ชุดคำสั่ง Ultralytics YOLOv8 จึงเก็บผลลัพธ์ในรูปแบบ List แต่อย่างไรก็ตาม ถ้านักศึกษาต้องการผลลัพธ์เฉพาะ object detection สามารถแก้ไขโค้ดบริเวณสีเหลืองเป็น output_img_path = os.path.join(output_path, img_name) และ result[0].save(output_img_path)



# หัวข้อ

- **Multi-Stage Detectors**
  - Region-based Convolutional Network (R-CNN)
  - Fast Region-based Convolutional Network (Fast R-CNN)
  - Faster Region-based Convolutional Network (Faster R-CNN)
- **Single-Stage Detectors**
  - You Only Look Once (YOLO)
- **การสร้าง YOLOv8 ด้วยชุดคำสั่ง Ultralytics**